Universitat Politècnica de Catalunya - BarcelonaTech

Artificial Intelligence and Cybersecurity in IIOT networks

Thesis supervisor

Linus Manubens-Gil

(Department of ***)

Degree

Master's degree in Machine Learning and Cybersecurity for Internet Connected Systems

Master's thesis

UPC

Abstract

The present thesis will analyze the impact of secure communication and anomaly detection within the Industrial IoT framework. At first, an evaluation of the additional overhead caused by the use of TLS for the communication via MQTT protocol on the Raspberry Pi 4 machine using the Mosquitto broker is conducted. Specifically, the performance overhead related to plain MQTT versus MQTTS in four different cryptographic settings: ECC/ECDSA, X25519/Ed25519, RSA (3072-bit), and RSA (4096-bit) is analyzed. TLS adds considerable overhead in terms of increased latency and CPU usage at the connection phase, whereas publishing has minimal overhead. Among the tested configurations, Ed25519 provides the best balance between security and performance, whereas RSA 4096-bit has the highest computational cost.
Second, the thesis proposes a hybrid intrusion detection framework that combines Decision Tree based feature extraction with lightweight neural network classifiers, including a Fully Connected model and several GRU variants. The approach was evaluated on the X-IIoTID and UNSW-NB15 datasets for both binary and multi-class classification. The results show that all models perform strongly in binary classification, with very close performance across architectures, achieved near-identical F1-scores exceeding 99.6% demonstrating that the Decision Tree leaf encoding alone provides highly discriminative features. In multi-class settings, the GRU-based models outperform the baseline, with DT-GRU H8 achieving a macro F1-score of 94.45% on X-IIoTID, while remaining computationally lightweight.
For unsupervised anomaly detection, an Isolation Forest model was trained exclusively on normal network traffic and evaluated across three datasets: X-IIoTID, UNSW-NB15, and a proprietary company IoT dataset, achieving overall accuracies of 85.29%, 93.45%, and 96.95%, respectively. A central contribution of this work is a data-driven dynamic threshold derived from the statistical distribution of anomaly scores on the normal training data, formulated as $\theta = \mu_s - k\sigma_s$. This approach eliminates the need for prior knowledge of attack proportions, which is impractical in real-world deployments, and introduces a tunable factor k that allows practitioners to explicitly control the trade-off between false positives and false negatives without requiring any labeled attack data.

Overall, the thesis shows that lightweight security mechanisms and efficient machine learning models can provide effective anomaly detection while maintaining practical performance for resource-constrained IoT and IIoT environments.

Acknowledgment

acknowledgment .

Acronyms

AI Artificial Intelligence

ANN Artificial Neural Network

CPU Central Processing Unit

CSR Certificate Signing Request

DDoS Distributed Denial of Service

DoS Denial of Service

DT Decision Tree

ECDSA Elliptic Curve Digital Signature Algorithm

ECC Elliptic Curve Cryptography

FC Fully Connected

OHE One-Hot Encoding

GRU Gated Recurrent Unit

IG Information Gain

IIoT Industrial Internet of Things

IoT Internet of Things

IP Internet Protocol

MAC Media Access Control

MQTT Message Queuing Telemetry Transport

MQTTS MQTT over TLS

RAM Random Access Memory

RNN Recurrent Neural Network

RSA Rivest-Shamir-Adleman

SSL Secure Sockets Layer

TCP Transmission Control Protocol

TCP/IP Transmission Control Protocol / Internet Protocol

TLS Transport Layer Security

UNSW-NB15 University of New South Wales Network-Based 15

X-IIoTID X-IIoT Industrial Intrusion Detection dataset

X25519 Elliptic Curve Diffie-Hellman key exchange over Curve25519

Ed25519 Edwards-curve Digital Signature Algorithm over Curve25519

Contents

Abstract 2
Acknowledgment 3
Acronyms 4
List of Figures 9
List of Tables 10

1 Objectives and Research Gap **1**

2 Background **3**
2.1 MQTT 3
2.1.1 Architecture of MQTT 3
2.2 Transport Layer Security protocol 6
2.2.1 Architecture of TLS 6
2.2.2 Security Limitations of MQTT 7
2.2.3 MQTT over TLS (MQTTS) 7
2.3 Commonly used asymmetric algorithms 9
2.3.1 RSA Cryptosystem 9
2.3.2 ECC Cryptosystem 9
2.3.3 X25519 9
2.3.4 Ed25519 10
2.4 Anomaly Detection 10
2.4.1 Types of anomalies 10
2.5 Unsupervised Anomaly Detection 12
2.5.1 Isolation Forest 12
2.5.2 Gated Recurrent Unit 15
2.5.3 Decision Trees and Entropy-Based Splitting 16

3 Related Work **17**

4 Methodology **18**
4.1 MQTT/MQTTS 18

4.1.1 Test of MQTT without TLS 18
4.1.2 Test of MQTT with TLS 19
4.1.3 Mosquitto Configuration and TLS Connection Testing 22
4.1.4 Data Collection and Statistical Analysis 22
4.2 Proposed Hybrid Models 23
4.2.1 Data Preprocessing 23
4.2.2 Decision Tree Feature Extraction 25
4.2.3 Neural Network Models 25
4.2.4 Training and Evaluation Procedure 26
4.3 UNSW-NB15 Dataset 28
4.4 X-IIoTID Dataset 29
4.5 Company Data Collection 30
4.5.1 Features extraction 30
4.6 Synthetic Anomaly Simulation 31
4.7 Isolation Forest for Unsupervised Anomaly Detection 34
4.7.1 Training Procedure 34
4.7.2 Hyperparameter Tuning 34
4.7.3 Thresholding and Evaluation 35

5 Results and Evaluation 37
5.1 MQTT/MQTTS Performance Evaluation 37
5.1.1 Latency Results 37
5.1.2 CPU Time Results 38
5.2 Evaluation Metrics 38
5.3 Hybrid models Performance 39
5.3.1 Binary Classification Results For X-IIoTID Dataset 39
5.3.2 Multi-class Classification Results For X-IIoTID Dataset 41
5.3.3 Binary Classification Results For UNSW-NB15 Dataset 44
5.3.4 Multi-class Classification Results For UNSW-NB15 Dataset . 45
5.3.5 Comparison with Related Work 47
5.4 IF model Performance 50
5.4.1 model Performance on X-IIoTID Dataset 50
5.4.2 Model Performance on UNSW-NB15 Dataset 52
5.4.3 Model Performance on Company Dataset 53

6 Conclusion and Future Work 55
6.1 Conclusion 55
6.2 Future Work 56

List of Figures

2.1 MQTT protocol architecture 4
2.2 Message flow of the MQTT CONNECT 5
2.3 TLS 4-way handshake 7
2.4 collective anomalies 11
2.5 contextual anomalies, Grey rectangle indicates anomaly 11
2.6 Point anomalies 12
2.7 Isolation Forest illustration 13

4.1 Hybrid Models Pipeline 23
4.2 Flow chart for classification. 24
4.3 Distribution of attack and benign labels in the UNSW-NB15 dataset 28
4.4 Statistics of observations included in the X-IIoTID dataset 29
4.5 class dis. 32
4.6 Violin plots of selected features illustrating the distribution differences between normal traffic and simulated anomalies (DoS/DDoS and data tampering). 33

5.1 Latency comparison for CONNECT and PUBLISH operations 37
5.2 CPU time comparison (with SD) for CONNECT and PUBLISH operations 38
5.3 FC_only Loss 40
5.4 FC_only Accuracy 40
5.5 DT+FC training and validation curves on X-IIoTID (binary-class) . . 40
5.6 DT+GRU training and validation curves on X-IIoTID (binary-class) 41
5.7 DT+FC training and validation curves on X-IIoTID (multi-class) . . 42
5.8 DT+GRU training and validation curves on X-IIoTID (multi-class) . 43
5.9 DT+FC training and validation curves on UNSW-NB15 (Binary-class) 45
5.10 DT+GRU training and validation curves on UNSW-NB15 (Binary-class) 45

5.11 DT+FC training and validation curves on UNSW-NB15 (Multi-class) 47
5.12 DT+GRU training and validation curves on UNSW-NB15 (Multi-class) 47
5.13 Score distribution with anomaly detection threshold (X-IIoTID) . . . 51
5.14 Score distribution with anomaly detection threshold (UNSW-NB15) . 53
5.15 Score distribution with anomaly detection threshold (Company Dataset) 54

List of Tables

4.1 TLS configurations used for the MQTT broker listeners. 21
4.2 DT configuration 25
4.3 Training hyperparameters for all models 27
4.4 Isolation Forest hyperparameters configuration 34

5.1 Binary classification results on the X-IIoTID (Macro) dataset with model complexity 39
5.2 Multi classification results on the X-IIoTID (Macro) dataset with model complexity 42
5.3 Binary cassification results on the UNSW15 (Macro) dataset with model complexity 44
5.4 Multi classification results on the UNSW15 (Macro) dataset with model complexity 46
5.5 Comparison between existing models and the proposed models on UNSW-NB15 dataset 48
5.6 Comparison between existing models and the proposed models on UNSW-NB15 dataset 49
5.7 Classification Performance Metrics (X-IIoTID) 50
5.8 Classification Performance Metrics (UNSW-NB15) 52
5.9 Classification Performance Metrics (Company Dataset) 54

Chapter 1

Objectives and Research Gap

This work focuses on addressing critical challenges in security and intelligent intrusion detection, while ensuring that the proposed solutions remain efficient and suitable for resource-constrained IoT environments.

First, this thesis evaluates the performance impact of Transport Layer Security (TLS) on MQTT communication by comparing plaintext MQTT with MQTT over TLS (MQTTS). The research gap lies in the fact that existing studies often provide only general comparisons between MQTT and MQTTS, without identifying which phases introduce the most overhead or consume the most resources. To address this gap, this work provides a detailed evaluation of both the connection (handshake) phase and the message publishing phase. It quantifies the overhead introduced by TLS in terms of connection latency, publishing latency, and CPU usage under identical experimental conditions. This provides a clear understanding of the performance overhead introduced by TLS and its implications for resource-constrained IoT devices.

Second, this thesis aims to develop a lightweight hybrid anomaly detection approach that combines Decision Tree-based feature extraction with neural network models. The research gap lies in the limited exploration of combining interpretable models with neural networks for improved feature representation. The proposed approach is designed to enhance detection performance while maintaining low computational complexity. In addition, many existing studies do not adequately consider model complexity, particularly in hybrid architectures, where increased complexity can limit practical deployment in resource-constrained environments.

Third, this thesis addresses one of the most significant challenges in unsuper-

vised anomaly detection methods, particularly in the Isolation Forest model. Most existing approaches rely on a contamination parameter, which assumes prior knowledge of the proportion of anomalous data—an unrealistic assumption in real-world industrial environments. To overcome this problem, this thesis proposes a dynamic thresholding derived from the statistical distribution of the data. This approach eliminates the need for any prior knowledge and introduces a tunable parameter that allows control over the trade-off between false positives and false negatives.

Chapter 2

Background

2.1 MQTT

The Message Queuing Telemetry Transport (MQTT) protocol is a lightweight protocol was created in 1999 with the goal of reducing the costs of transmission it widly for the Internet of Things (IoT),works on the top of transmission control protocol/internet protoco TCP/IP , with Mqtt devices can send message(puplish) to the central servar called broker and broker can send this message to the other devices that subscribe in the same topic [1]. Mqtt ability to operate under challenging communication conditions with low data consumption makes it popular in industrial systems . Now Mqtt becomming one of the main protocol in the IOT because

- QoS Data Delivery
- minimum requirements for power
- QoS Data Delivery
- simplicity of implementation

2.1.1 Architecture of MQTT

The description of this section according to MQTT v3.1 [1]. From architecture view, MQTT is a message-oriented protocol based on the publish/subscribe model: the two main components are the client and the broker, which publish, and another client subscribes to the same topic through the broker server. in simaple words, the client sends message to the broker, and the broker is responsible for getting all the messages from from all the client and filtering them based on the topic, and then publishing the message to the client that asked for the same topic

the tobic as text name (UTF-8) can have many levels separated by a slash (\) Each topic at least contains one character, and the client can create new topic when starting to publish a message on it without needing to per configuration, some topics start with reserved symbol, which is for internal statistics for the broker; the client can't publish on this topic . The Figure 2.1 is a simplified architecture. of MQTT

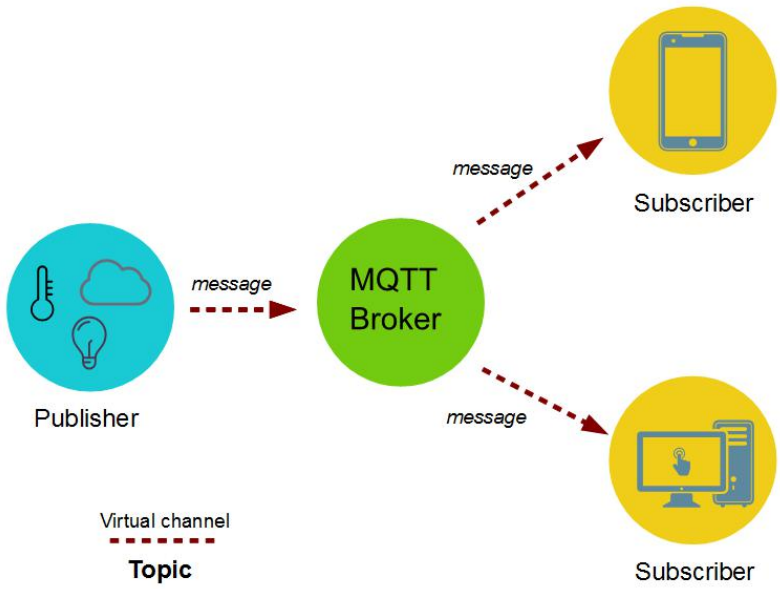


Figure 2.1: MQTT protocol architecture

One other feature of the MQtt is the quality of service Qos (quality of service) level. It is the sender and the receiver regarding ensuring message delivery safely,

and there are three levels :

- QoS 0: the message It arrives once or not at al , it is consider fastest mode to transfer this also called fire and forget.
- QoS 1 the message arrive once at least when the sennder pupish to the broker send PUBACK to the sender(puplisher) mens that the messsage has been recived and if this PUBACK does not recive to the puplish with certin time it will send again same message and set a flag in the message DUP = true that refers the message is duplicate and not the orginal . in this lavel the puplisher insure that the message does not loss whem the nwtorks get lost
- QoS 2 the message arrive exactly once without duplication or loss. This method is safe but slower compared to the previous Qos levels . to insiure the the delever only one the broker and puplisher 3 stage :

- First stage (PUBREC): once the broker receives the message from the publisher, it sends PUBREC as confirmation.
- Second stage (PUBREL): after receiving PUBREC, the publisher sends PUBREL to the broker as confirmation.
- Third stage (PUBCOMP): after receiving PUBREL, the broker sends PUBCOMP, the final notification to the publisher that the message was received correctly.

2.1.1.0.1 CONNECT

it is a packet that the cliend send to broker to intial the connection and it contain some fields :

- ClientId : repersent uniqe client ID (UTF-8) for each customer at the broker.
- KeepAlive : the client send PING Request to the broker in eshculd time and the broker replyed with Ping Response to insure the connection is still alive
- CleanSession : it is a flage between 0 and 1 determin whether the broker store subscriptionand of the clients anll missed messages (set to 0) or Nothing is stored(set to 1)

thes are mondetory fields and in addtion to this fields there are optinal field include passwords and username that insure that the client is autnticated the password send as a normal plaintext another optionl fields what called Will Message it is a message send the subscribers that notifiyed otgers cliend when the cliend abnormally disconnected affter the client send CONNECT message the broker response with CONNACK it esstablish connectin . As illustration in Figure 2.2

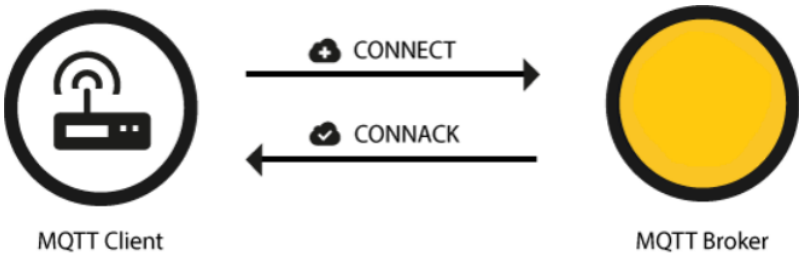


Figure 2.2: Message flow of the MQTT CONNECT

2.1.1.0.2 SUBSCRIBE

is a client that listens one or more topic from the broker. The broker delivers messages to all clients subscribed to matching topics and this command contain client

ID and quality of service(Qos) and list of tobics .affter the resive subscribe message the broker respose witth SUBACK acnulegment message that says theSubscription is successful.

2.1.1.0.3 PUBLISH

a client publish messages immediately when it connects to a broker and then send to all subscribers that shard on one topic this message contain the topic name and payload the actual message that wann send that can be text, numbers, images, binary data or JSON. Aso Qos can set for each messsage To determine guaranteed to reach the other party. if the Qos not 0 the message Packet Identifier flage use to folaw the message via the flow . also tthere are DUP flag show s the message if it was duplicated vrsion or no .

2.2 Transport Layer Security protocol

The section will provide an overview of TLS and the basic cryptographic concept. TLS is a widely used protocol that ensures privacy and data integrity between two communicating parties over TCP/IP. It is essentially an improved version of the SSLv3 protocol, and it is defined in RFC 2246 [2]. The latest standard version is TLSv1.3. one of the most comman used of TLS is in HTTPS where encrept credential , personal data and othere protects sensitive information . although the ssl/TLS is orginaly designed for for issure related to WWW, TLS now widly used to secure many application-layer protocols . To undstand how TLS achive hight security its important to interduse concept of puplic key creptography the puplic key used two pairs privet an public key , the priviet key only the owner know and the puplic key is its known for every one , thesis keys pupic and privet are corekated but puplic key does not revel any info about the privet key , the puplic key used to encerpt the message while privet key use to decrepte the message For more details [3] [4]

2.2.1 Architecture of TLS

The Transport Layer Security provides secure communication over a network. It is designed to ensure "privacy and data integrity between two communicating applications." During the handshaking process, "the client sends a ClientHello message, to which the server must reply with a ServerHello message." This step is carried out in order to determine the required encryption parameters, like protocol version

and cipher suites. The server sends its certificate to be authenticated, and a key exchange is performed to create a shared secret, and all subsequent communication is encrypted using the established keys after the exchange of Finished messages [5].

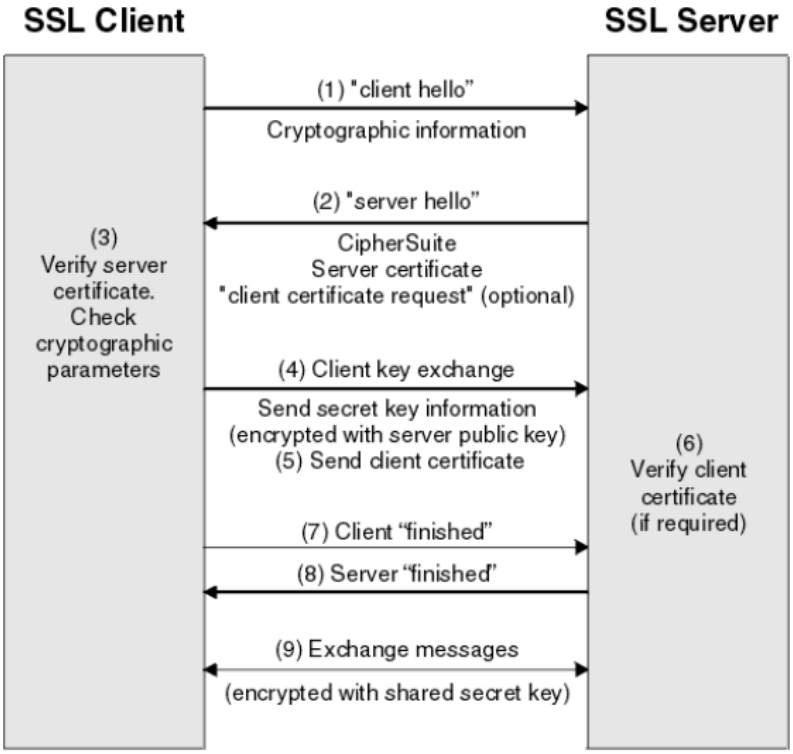


Figure 2.3: TLS 4-way handshake

2.2.2 Security Limitations of MQTT

widely adopted in Internet of Things (IoT) environments due to its lightweight design and efficiency. However, MQTT by default does not provide any built-in security mechanisms, which makes it vulnerable to attacks such as credential theft and message interception. When operating without Transport Layer Security (TLS), authentication is limited to the use of a username and password transmitted in plain text within the CONNECT packet[1]. This exposes IoT systems to man-in-the-middle (MITM) attacks, where adversaries can easily sniff traffic and obtain sensitive information. Furthermore, data privacy and integrity are not guaranteed, as published topics and payloads can be intercepted or modified without detection.

2.2.3 MQTT over TLS (MQTTS)

MQTT combined with TLS significantly enhances the security of communication. TLS provides encrypted channels , ensuring confidentiality of exchanged data and

protecting authentication credentials through its handshake mechanism. It also introduces integrity checks via cryptographic hashes, which prevent undetected tampering of messages[2]. Nevertheless, these security improvements come at a cost. The TLS handshake adds considerable overhead in terms of connection establishment time, bandwidth consumption, and computational requirements. Devices operating with TLS consume more power and experience higher latency compared to plain MQTT. This trade-off is particularly relevant for resource-constrained IoT devices, where limited memory and processing capacity may make TLS adoption challenging[3].

2.3 Commonly used asymmetric algorithms

2.3.1 RSA Cryptosystem

The RSA cryptosystem, introduced in 1977 by Ron Rivest, Adi Shamir, and Leonard Adleman, is one of the most widely used asymmetric encryption algorithms [6]. Its security is based on the computational difficulty of the integer factorization problem.

RSA operates using a pair of keys: a public key (N, e) and a private key d, where N is the product of two large prime numbers. A message M, represented as an integer such that $M < N$, is encrypted using modular exponentiation:

$$C = M^e \bmod N \tag{2.1}$$

The ciphertext C can be decrypted using the private key d as follows:

$$M = C^d \bmod N \tag{2.2}$$

Due to its strong security properties, RSA is widely used in secure communication protocols such as SSL/TLS, digital signatures, and key exchange mechanisms [7].

2.3.2 ECC Cryptosystem

Elliptic curve cryptography is based on hard mathematical problems defined over elliptic curves. Its application to public-key cryptography was independently proposed by Miller [8] and Koblitz [9]. These cryptosystems rely on the algebraic structure of the group of points on an elliptic curve defined over a finite field. In particular, the security is associated with the difficulty of solving the elliptic curve discrete logarithm problem, which is considered more intractable than its classical counterpart. This property makes elliptic curve-based systems attractive for secure communications.

2.3.3 X25519

X25519 is a Diffie-Hellman key agreement function based on Curve25519, introduced by Daniel J. Bernstein in 2006 [10]. It operates on a Montgomery curve, which allows for efficient scalar multiplication and runs in constant time regardless of input values, making it inherently resistant to timing-based side-channel attacks. it has been adopted as the key exchange mechanism in TLS 1.3, [11] making it particularly suitable for resource-constrained IoT devices.

2.3.4 Ed25519

Ed25519 is a digital signature scheme , introduced by Bernstein et al. in 2011 [12]. It uses the EdDSA construction, which provides deterministic signature generation without requiring a random number generator during signing a critical security advantage, as weak randomness in schemes like ECDSA can lead to full private key exposure. Ed25519 produces compact 64-byte signatures and achieves high signing and verification speeds, while incorporating strong resistance to side-channel attacks [12].

2.4 Anomaly Detection

Anomaly detection is the process of identifying rare events, unusual data points,or outliers that deviate significantly from the norm in a dataset. These anomalies can signal potential issues or interesting patterns in the data. Anomaly detection as a topic is well-researched, with many publications in both statistics and data science. The real challenge is to define and construct the right data model in order to separate outliers from the actual normal data.

2.4.1 Types of anomalies

there are three main categories of anomalies can be classified , each of which are explained in more detail in the following subsections.

- Contextual anomalies
- Collective anomalies
- Point anomalies

2.4.1.1 Contextual anomalies

as the name, it appears when looking at a group of points together, but it does not appear when observing each point in the data individually.As is shown in Figure 2.5, the Grey rectangle indicates an anomaly.

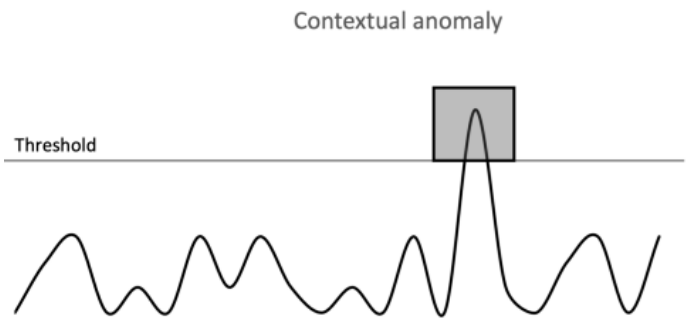


Figure 2.4: collective anomalies

2.4.1.2 Collective anomalies

A contextual anomaly refers to an instance that becomes anomalous only within a specific context. In other contexts, the same instance shows no indication of abnormal behaviour. Identifying this type of anomaly relies on combining both contextual and behavioural features. Contextual features typically relate to the most often time and/or space of the instance, while behavioural features depend on the specific domain in which they are applied For example,the data transfer rate may appear unusually high, indicating abnormal behavior, looking on the same date over several years, it may shows this increase occurs repeatedly every year, making it part of a normal pattern.

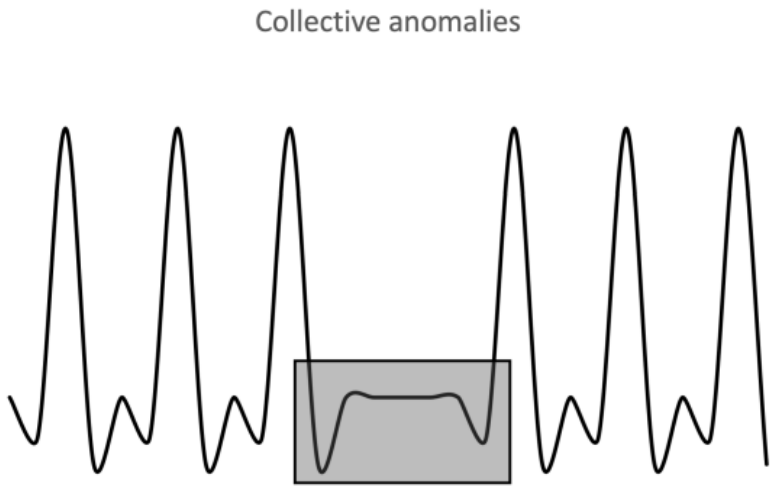


Figure 2.5: contextual anomalies, Grey rectangle indicates anomaly

2.4.1.3 Point anomalies

point anomaly is a single data point that deviates significantly from other points in the dataset. It represents these anomalies as extremely irrelevant across all datasets. For example, a sudden spike in the data transmission rate, much higher than usual, without any prior context, would be considered a point anomaly. An example illustrates in Figure 2.6

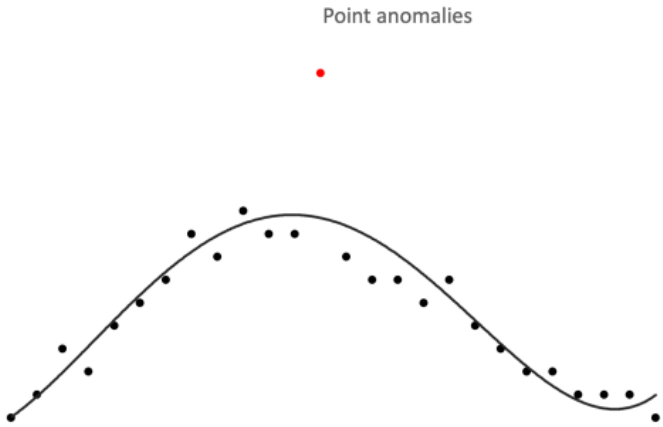


Figure 2.6: Point anomalies

2.5 Unsupervised Anomaly Detection

2.5.1 Isolation Forest

Isolation Forest is an ensemble model using machine learning models, which are light-weighted and are quickly trained used to find unusual data points it isolates anomalies by separating them from normal data points **sklearnIF**. It builds extremely randomized decision trees as base estimators, which makes it well suited for anomaly detection. it computes an anomaly score defined as [13]

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$

Where $h(x)$ $h(x)$ is the number of edges in the tree for the point x, returns the path length of x

and $c(n)$ is the normalization constant for a sub-sample of size n.

$E[h(x)]$ is the average path length across all trees.

and $s(x, n)$ represents the anomaly score assigned to the data point x.

The normalization constant $c(n)$ is defined :

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}$$

where $H(i)$ is the harmonic number and it can be estimated by:

$$H(i) = \ln(i) + \gamma$$

and γ is Euler's constant $\gamma \approx 0.5772156649$.

As the average number of edges required to isolate a data point approaches zero, the anomaly score increases, meaning anomalies are easier to separate from normal data [14]. Introduced in 2008 and integrated into Scikit-learn in 2016 [15], the Isolation Forest is built upon an ensemble of randomized trees and can operate with or without replacement (default is without). Both the attribute to split and the split value are chosen randomly, which helps avoid overly greedy tree construction. As illustration in Figure 2.7

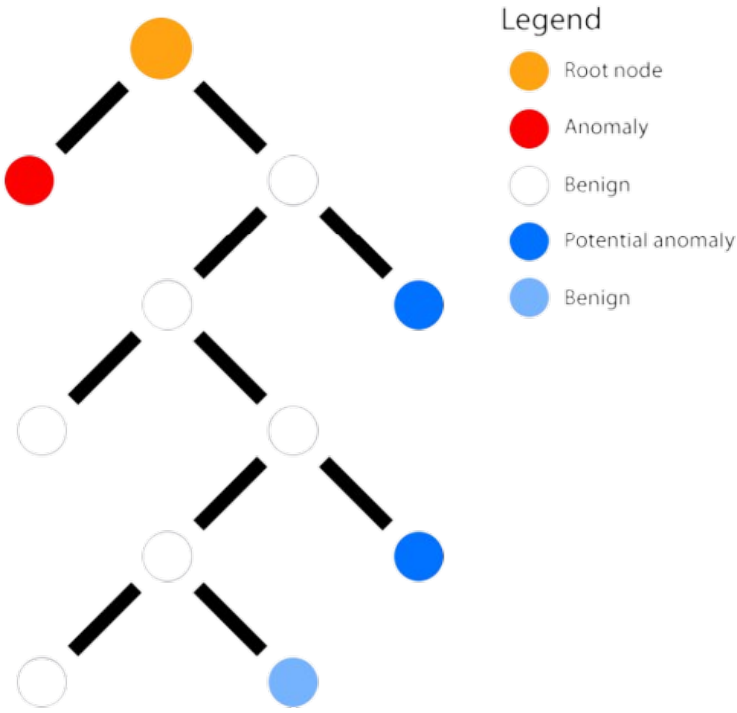


Figure 2.7: Isolation Forest illustration

The anomaly, shown as a red node, becomes clear because it is the only node on the left side of the cutoff value at the root. After that split, the leaf nodes on the right side (except for those on the lowest level) suggest possible anomalies. The nodes at the bottom level are treated as normal, including the one on the far right. Although that node might usually be considered anomalous, it is labeled as normal here simply because there are no more nodes to be distributed .

Key advantages of Isolation Forest [13]:

- Scalability : Trees can be built in parallel easily and independently, making it suitable for large-scale data.
- high-dimensional data : Ability to handle high-dimensional data with many features as it relies on random partitioning instead of densities or distances.
- global anomalies : anomalies are few and different and the algorithm can detect these anomalies. Each tree isolates anomalies in a different way. When the forest results are combined, anomalies are effectively detected.
- Robustness and low sensitivity to outliers : outliers always appear with less depth, and it is a shorter paths within the trees.

2.5.2 Gated Recurrent Unit

The Gated Recurrent Unit (GRU) was introduced by Cho et al. (2014) [16] as a simplified recurrent neural network architecture designed to improve the modeling of sequential data and to solve the common problems of vanishing and exploding gradients in traditional RNNs [17]. It is a simplified version of the Long Short-Term Memory (LSTM) network, which reduced computational complexity with fewer gating mechanisms. It uses 2 gates rather than 3, having less operations, namely the reset gate and the update gate, to control the flow of information through the hidden state. The reset gate determines how much past information is ignored, while the update gate controls how much of the previous hidden state is retained. By combining the current input with the past hidden state through a candidate activation, GRU is able to capture long-term dependencies while maintaining computational simplicity.

Reset Gate:

$$r_t = \sigma(W_r X_t + U_r h_{t-1} + b_r)$$

- If $r_t \approx 0$, the past state is mostly ignored.
- If $r_t \approx 1$, the past state is mostly kept.

Candidate Hidden State:

$$\tilde{h}_t = \tanh(W_h X_t + U_h(r_t \odot h_{t-1}) + b_h)$$

- r_t controls how much of the past hidden state is used.
- If $r_t = 0$, the past is completely ignored (acts like starting fresh).
- If $r_t = 1$, it behaves more like a regular RNN.

Update Gate:

$$z_t = \sigma(W_z X_t + U_z h_{t-1} + b_z)$$

- $z_t \approx 0 \rightarrow$ Keep most of $\tilde{h}_t$ (new information dominates).
- $z_t \approx 1 \rightarrow$ Keep most of h_{t-1} (old memory dominates).

Hidden state computation:

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$$

- z_t blends old and new information smoothly.
- If $z_t \approx 1$, then $h_t \approx h_{t-1}$ (carry over memory, no major update).
- If $z_t \approx 0$, then $h_t \approx \tilde{h}_t$ (new information replaces old).

2.5.3 Decision Trees and Entropy-Based Splitting

A Decision Tree is a supervised machine learning model used for classification and regression tasks. It partitions the input space into regions by recursively splitting the data based on feature values, forming a tree-like structure of decision rules.The algorithm continues to split the data, creating new nodes and branches, The algorithm continues to split the data, creating new nodes and branches, until predefined stopping conditions are met.
One of the most common criteria used to determine the quality of a split is entropy, which measures the impurity or uncertainty of a dataset. For a classification problem with K classes, entropy is defined as:

$$H = -\sum_{i=1}^{K} p_i \log_2(p_i) \tag{2.3}$$

where p_i is the proportion of datapoints in class i, K is the number of classes.

To select the best split, Decision Trees use Information Gain (IG), how much the entropy decreases after a split:

$$IG(A) = H(S) - \sum_{j=1}^{m} \frac{n_j}{n} H(S_j) \tag{2.4}$$

where $H(S)$ is the entropy of the parent dataset, S_j represents the j-th subset resulting from the split on attribute A, n_j is the number of samples in subset S_j, n is the total number of samples in the parent node, and m is the number of subsets. A

higher Information Gain means the feature provides a better split, leading to purer child nodes with less uncertainty (lower entropy) [18].

Chapter 3

Related Work

Chapter 4

Methodology

4.1 MQTT/MQTTS

In this section, describe the methodology used to evaluate the performance of MQTT and MQTTS. The goal is to evaluate the impact of TLS on system performance. This experiment was conducted on Raspberry Pi 4 GB, with Mosquitto as a broker, and it was configured to listen on 1883 port, with no secure communication, and multiple TLS-secured ports for encrypted communication; all ports were enabled at the same time, so each test was repeated easily under the same connection to measure the performance of encryption and non-encryption MQTT to compare performance metrics between encrypted and unencrypted. This comparison is essential to understand the security encryption overhead in the IoT devices. The evaluation focuses on both connection performance and message publishing performance.

4.1.1 Test of MQTT without TLS

To evaluate the baseline performance of the MQTT protocol without encryption, a set of experiments was carried out using a Python-based MQTT client running on a Raspberry Pi 4 , to measure latency and CPU usage during MQTT communication without a TLS security layer.
The results provide a reference point for later comparisons with MQTT over TLS (MQTTS) to quantify the overhead introduced by encryption. The experiments were carried out using the Paho MQTT library, which enables to connect to an MQTT broker to publish messages. The MQTT broker was Mosquitto, running locally on the same Raspberry Pi. The broker was configured to use the default non-secure MQTT port (1883) for plaintext communication, with authentication enabled through a username and password. On the client side, the Python library

`psutil` was used to track CPU time, providing real-time insight into system resource consumption. CPU consumption was determined from the process CPU time reported by the operating system.
Connection performance was evaluated by measuring the time required to establish a connection with the broker. Before connecting, the client recorded the initial timestamp and CPU time. It then sent a CONNECT packet to the broker.Upon receiving a CONNACK packet, confirming a successful connection, a connection callback function recorded the final timestamp and CPU time. Connection latency was calculated as the difference between before sending CONNECT and receiving CONNACK. CPU consumption during connection was calculated as the difference between CPU times measured before and after the connection.
Once connected, the publishing performance was tested by sending twenty MQTT messages to a predefined topic. Each message included a timestamp in its payload. The experiment used QoS 1, which ensures that the broker acknowledges each message with a PUBACK packet. For each message, the client recorded the start time immediately before sending the PUBLISH packet and the end time upon receipt of the corresponding PUBACK acknowledgment. Message latency was computed as the difference between these timestamps.
During publishing, CPU time was monitored for each message. CPU consumption was calculated as the difference in process CPU time measured immediately before sending the PUBLISH packet and after receiving the PUBACK acknowledgment from the broker. A short delay was inserted between messages to maintain stable measurement conditions.

4.1.2 Test of MQTT with TLS

In this section, we evaluate the performance of the MQTT protocol when encrypted with TLS. The tests conducted are the same as those in the previous section but focus on measuring latency and CPU usage during MQTT communication with TLS security. These results allow for a comparison between different encryption algorithms and key sizes, showing their impact on the performance of resource-constrained IoT devices.
The experiments were conducted on the same Raspberry Pi 4 and Mosquitto broker setup described in the previous section, with the addition of multiple TLS-secured listeners using different encryption algorithms. The experimental procedures followed the same steps as the MQTT tests without TLS to ensure comparable conditions.

Four TLS configurations were used in the experiments:

- **ECC/ECDSA TLS:** Elliptic Curve Cryptography
- **X25519/Ed25519 TLS :** Modern elliptic curve algorithms
- **RSA 3072-bit TLS:** RSA with a 3072-bit key
- **RSA 4096-bit TLS:** RSA with a 4096-bit key

All certificates and certificate authorities used in the setup were self-signed.

Prior to the experiments, each broker listener's TLS setup was validated using OpenSSL:

```
openssl s_client -connect host:<port> -showcerts
```

The TLS configurations for each port were as follows:

Port	**Configuration**	**Certificate/Auth**	**Key Exchange**	**TLS Cipher**
8884	ECC/ECDSA	ECDSA P-256 (256-bit)	X25519	AES-256-GCM
8886	Ed25519	Ed25519 (256-bit)	X25519	AES-256-GCM
8887	RSA 3072	RSA (3072-bit)	X25519	AES-256-GCM
8888	RSA 4096	RSA (4096-bit)	X25519	AES-256-GCM

Table 4.1: TLS configurations used for the MQTT broker listeners.

All configurations employed TLS 1.3 with AES-256-GCM and SHA-384 as the hash function. The key exchange method was consistently X25519.
Connection performance with TLS was evaluated by measuring the time required to establish a secure connection with the broker. After establishing the connection, the client published twenty messages to a specific topic using QoS level 1. Message latency and CPU consumption were measured following the same procedure described in the previous section, with the addition of TLS overhead.

4.1.3 Mosquitto Configuration and TLS Connection Testing

The Mosquitto configuration file (`mosquitto.conf`) modified in order to enable both the unencrypted listener (port 1883 for plain MQTT) and encrypted listeners (TLS-enabled ports). Before modifying the configuration, the certificates were moved to the Mosquitto directory and correct permissions were applied. These certificates were used by Mosquitto to authenticate incoming clients and to encrypt communication using the TLS handshake. Before running the performance tests, several tests were conducted to confirm that TLS communication was working correctly These tests included:

- Subscribing to a topic using TLS
- Publishing messages using TLS
- Verifying the TLS session and checking the cipher suite using OpenSSL

4.1.4 Data Collection and Statistical Analysis

After completing all latency and CPU tests, the collected data were organized and analyzed. The analysis included the following steps:

- Calculating the mean, minimum, maximum, and median values for latency.
- Calculating average CPU consumption for each operation.
- Comparing secure vs. non-secure results to understand the overhead introduced by TLS.
- Plotting graphs to visually compare latency and CPU.

These results are presented in the Results section, where the performance differences between MQTT and MQTTS are discussed.

4.2 Proposed Hybrid Models

This section illustrates a hybrid architecture that combines a Decision Tree (DT) as a feature extractor with lightweight neural networks classifiers, including a Fully connected (FC) model and several Gated Recurrent Units (GRU) variants to perform multi-class and binary intrusion detection classification on different datasets. The main objective is to imperove classification performance by transferring the original feature info to a more informative representation before applying neural network models, the pipeline follows with sequential stage: data preprocessing, DT-based feature extraction, and neural network classification.

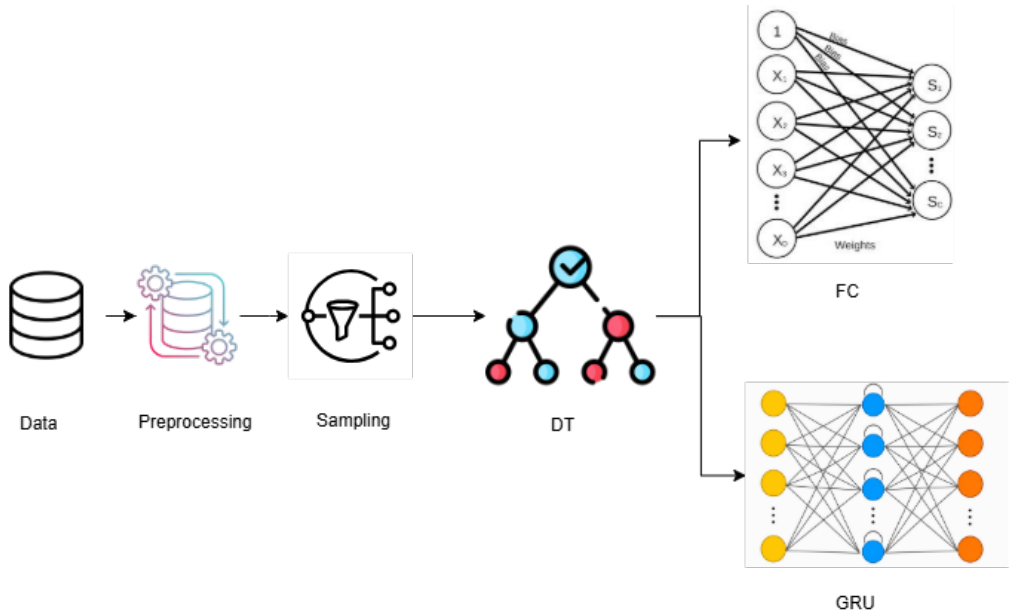


Figure 4.1: Hybrid Models Pipeline

4.2.1 Data Preprocessing

Data preprocessing was applied to the datasets used in this work. Raw network traffic data contains diverse column types, numeric and categorical features, all of which need to be prepared properly before using them in a machine learning model so the model can process them consistently. The dataset is loaded from a CSV file and was divided into training, validation, and testing sets, using a stratified splitting to preserve class distribution specifically 20% of the full set is applied as the test set and remaining 80% is further divided so that 20% of the portion as validation set and leaving the 64% of the data for training. Next, the type of each column is

identified. Date/time columns are converted into numeric values (Unix time). For string columns, check if they can be converted into numbers. If yes, they are treated as numeric; otherwise, they are treated as categorical. Categorical features were encoded using label encoding, where each category is mapped to an integer value based only on the training data, and the same mapping is applied to validation and test sets. Missing values in numerical features were handled using median imputation, where each missing value was replaced with the median computed from the training set. For categorical features, missing values were first replaced with a placeholder value before encoding, allowing them to be treated as a separate category during label encoding. Then, across all the features, Min-Max normalization was used to scale all values.

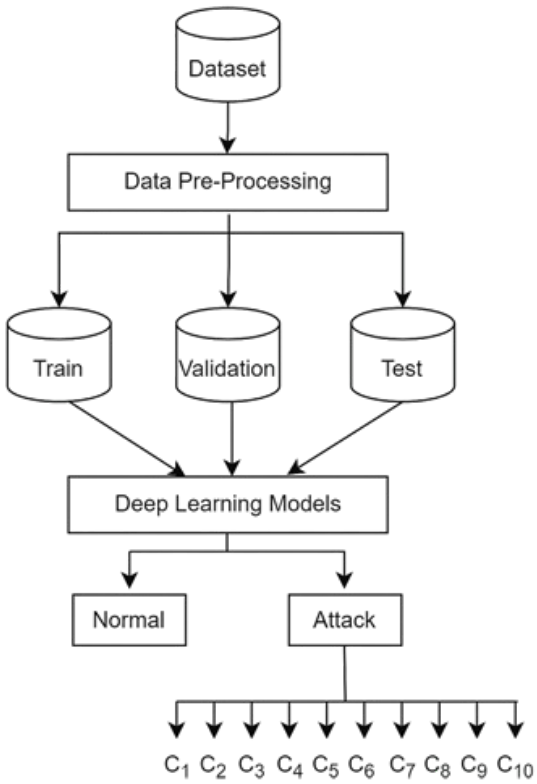


Figure 4.2: Flow chart for classification.

4.2.2 Decision Tree Feature Extraction

A Decision Tree was trained on the preprocessed dataset and used as a feature extractor. The DT can learn complex, non-linear relationships between raw features in an efficient and computationally effective manner. Each data sample ends up in a leaf node position, giving a meaningful representation of the data. The model was configured using entropy to split data, with a maximum depth of 12, a minimum of 20 samples per leaf node, and a minimum of 2 samples required to perform a split. The best splitter strategy was used to determine optimal splits.

The Decision Tree parameters:

Parameter	**Value**
class_weight	None
criterion	"entropy"
max_depth	12
max_features	None
min_samples_leaf	20
min_samples_split	2
splitter	"best"

Table 4.2: DT configuration

After training, each sample is passed through the Decision Tree to determine the leaf node it reaches. The resulting leaf indices are then transformed into a high-dimensional representation using one-hot encoding, that are used as input to the neural network models.

4.2.3 Neural Network Models

The transformed feature vectors generated by the Decision Tree were used as input to differentiate Neural Network Models and output class-probability scores for the same set of labels. Two types of neural network architectures are: a fully connected simple dense layer and Gated Recurrent Unit models, which are described in the next sections.

4.2.3.1 Fully Connected Model

The FC model is the simplest neural network classifier with a single linear layer. The OHE leaf features from DT were fed into a single linear layer, and then maps these input features vector to the output classes.

4.2.3.2 GRU-Based Models

The GRU introduced a more advanced architecture that can capture complex patterns. several GRU configurations were implemented with different hidden sizes, including 8, 16, and 32 units. The only difference is the size of the hidden layer, while the overall structure remains the same. The feature vectors from the Decision Tree were reshaped into sequences of length one using an unsqueeze operation to match the GRU input format. The GRU uses a single layer with one direction, and the output from the hidden state is passed to a linear layer to produce the final class predictions.

4.2.4 Training and Evaluation Procedure

All models were trained under the same conditions for future comparison. The training was implemented using PyTorch with Adam optimizer and a fixed learning rate.

During training, the model handles the dataset in small batches of data. For each training epoch, the model is set to training mode and looped through each batch of the training DataLoader. For every batch, a feedforward computation is performed, the error is calculated by comparing predictions with the ground truth, gradients are computed via backpropagation, and the model adjusts its parameters using the optimizer.

After each training epoch, validation loss and performance metrics are computed over the entire validation dataset, and early stopping is applied based on validation loss. The model state with the lowest validation loss is saved, and training stops early if no improvement is seen for a set number of epochs. The number of trainable parameters is recorded for each model to enable a comparison of model complexity.

Testing / Evaluation Metrics The optimal network is then assessed on unseen data. Performance is measured using:

- Proportion correct (accuracy)
- Class-averaged F1 (Macro F1 score)
- metrics (precision, recall, and F1 per class)

Finally, training performance plots, including validation loss and validation accuracy, are used to visualize the learning process.
The training hyperparameters used are shown in Table 4.3

Hyperparameter	**Value**
Batch size	256
Maximum epochs	20
Learning rate	1×10^{-3}
Optimiser	Adam with weight decay 1×10^{-5}
Loss function	Cross-entropy
Gradient clipping	Max norm of 1.0
Early stopping	3 epochs

Table 4.3: Training hyperparameters for all models

After training, the best model is evaluated on the test set. Performance is measured using accuracy, macro F1-score, and per-class precision, recall, and F1-score In addition, validation loss and accuracy curves are plotted to visualize the learning process.

4.3 UNSW-NB15 Dataset

The UNSW-NB15 dataset is a relatively new and general dataset for network traffic that encompasses normal and anomalous behaviours. It was derived from ACCS, and the set data are real-world attack data, which make it a valuable tool for assessing IDS. The proposed dataset encompasses several features which captures traffic characteristics pertinent to the classification of attacks in an automated system. These traffic features measured at a given period, gives understanding on how a network behaves in normal and attack situations. The datasets were in CSV format and can be downloaded directly from the UNSW website or from some research papers that relate to network security and anomaly detection research. These datasets are large and well-ordered, which allows constructing and verifying intrusion detection models on their basis. The UNSW-NB15 dataset comprises of 49 features where the output variable is attack type. The distribution of labels within the UNSW-NB15 dataset is shown in Figure 4.3. Approximately 53% of the communication is labeled as benign, representing normal network activities. Among attack types, exploits account for the largest proportion, followed by DoS attacks and fuzzers. Other categories, such as reconnaissance, generic, backdoor, and analysis, are present in smaller yet significant quantities. Some categories, like shellcode, back doors, and worms, represent a small fraction of the dataset [19].

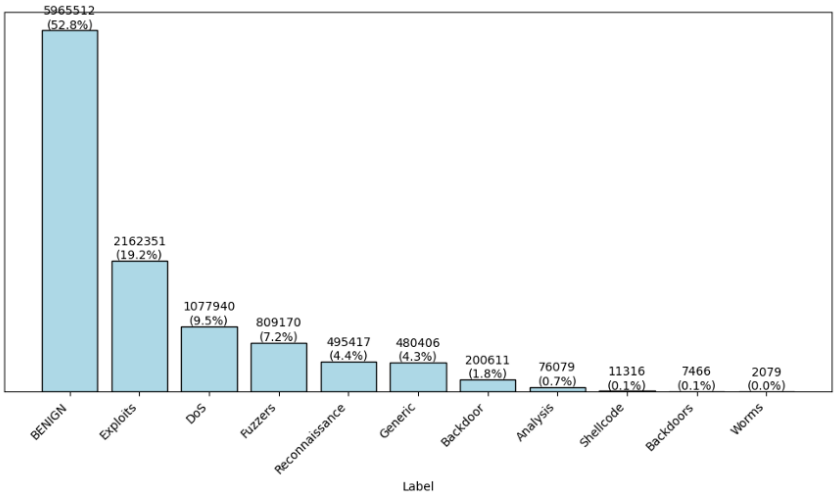


Figure 4.3: Distribution of attack and benign labels in the UNSW-NB15 dataset

4.4 X-IIoTID Dataset

The X-IIoTID dataset is an up-to-date dataset developed for the purpose of detecting intrusions in complex Industrial Internet of Things (IIoT) networks. It is designed to reflect real-world IIoT environments, where multiple heterogeneous devices operate together within a large-scale and complex structure.

This dataset was created by collecting data from devices of different types and brands, as well as from various platforms to which these devices are connected. It adopts a comprehensive approach by integrating multiple sources of information within the IIoT network.

The X-IIoTID dataset consists of traffic information and state changes obtained from devices and communication protocols within the network. In addition, it includes logs, resource-related data, and alert features collected from different devices and network links. The dataset contains 68 features and a large number of samples, making it suitable for evaluating deep learning-based intrusion detection systems.

The dataset includes nine main categories of intrusions, namely Reconnaissance, Command and Control, Weaponization, Exploitation, Tampering, Lateral Movement, Exfiltration, Crypto Ransomware, and RDoS, in addition to normal traffic. Each of these categories contains multiple sub-types, resulting in a total of eighteen different intrusion types.

Due to its comprehensive structure, high dimensionality, and representation of modern attack scenarios, X-IIoTID is considered a well-defined dataset for evaluating intrusion detection systems in complex IIoT environments [20].

Attack class #1	Attack class #2	Number of instances	Total number of instances
Reconnaissance	Generic Scanning	50277	127590
	Scanning vulnerability	52852	
	Discovering resources	23148	
	Fuzzing	1313	
Weaponization	Brute-force	47241	67260
	Dictionary	2572	
	Insider malicious	17447	
Exploitation	Reverse shell	1016	1133
	MitM	117	
Lateral Movement	Modbus-register reading	5953	31596
	MQTT-cloud broker subscription	23524	
	TCP Relay	2119	
Command &Control	Command &Control	2863	2863
Exfiltration	Exfiltration	22134	22134
Tampering	False data injection	5094	5122
	Fake notification	28	
Crypto Ransomware	Crypto Ransomware	458	458
RDoS	RDoS	141261	141261
Normal	Normal	4211417	421417

Figure 4.4: Statistics of observations included in the X-IIoTID dataset

4.5 Company Data Collection

The section illustrates the data collected in the IoT environment. MQTT traffic was captured on the Raspberry Pi between the default port 1883 and the broker by using tshark, which is the command version of Wireshark, and specific filters were applied to exclude certain message types, including PINGREQ (the client sends this message to the broker to check the connection, which tells the broker the client is still alive) and PINGRESP (the broker replays the client indicated the broker is still alive), these messages do not contain payload data; it's only to keep the connection. The captures are saved into JSON, which contains fields such as timestamps, TCP/IP headers, and MQTT payload. A JSON format was chosen to save the data because this format is suitable for programmatic extraction and further analysis in the next stage. The captures take several days; each day captures more than seven hours.

4.5.1 Features extraction

After collecting the JSON files, transfer them to an external PC to merge them and extract the offline features. Python is used to perform this process due to its flexibility and the availability of the Robust library. The following features were extracted:

- frame_time_delta: represents the time interval between one packet and the previous packet.
- tcp_len: length of the TCP packet (payload), tells how much actual info is in the TCP packet.
- cp_window_size: the TCP window size, which means how much data can be sent before waiting for an acknowledgment.
- mqtt_msgid: MQTT assigns a unique identifier to the messages.
- mqtt_len: This is the total length of the MQTT message, containing headers and payload (size of the MQTT message is in bytes).
- mqtt_msgtype: represents the MQTT message type.
- IP address of the sender and the MAC address of the device.
- Sensor values: were extracted when decoding the hexadecimal payload of the MQTT PUBLISH messages.

- Publish rate: represents the number of messages sent by each client.
- Frame_count: represents the number of packets sent by client.

The final dataset, containing metadata, source information, sensor data, and dynamic traffic features, was saved in CSV format for further analysis.
The network environment was monitored, and the unique IP address and MAC address of devices crossing the network were recorded. After the collection was complete, all recorded IP and MAC addresses were verified to ensure that they belonged to company devices.

4.6 Synthetic Anomaly Simulation

To evaluate the anomaly detection model, synthetic anomalous data were generated and combined with real normal IoT traffic. The simulation process is based on the statistical properties of the data and on known IoT attack behaviors described in [21]. Two attack types were Simulation Denial of Service/Distributed Denial of Service and Data Tampering. These two types were chosen because they represent fundamentally different threat models, one volumetric and network level, the other on the application-level.

All attack samples were generated statistically from the real normal traffic. The mean and standard deviation of every numeric feature were first computed from the collected data, and these baseline statistics were then used to produce synthetic samples. The final dataset consists of normal traffic combined with DoS/DDoS and Data Tampering instances.

The DoS/DDoS anomaly represents abnormal traffic behavior. To simulate this, key traffic features were modified. Specifically, the frame count was significantly increased, the time between packet transmissions was reduced, and the publish rate was increased. These changes together create a high-volume traffic pattern that clearly differs from normal operation. At the same time, other features were kept within their normal ranges to maintain realistic data characteristics. In addition, random IP and MAC addresses were generated to simulate distributed attack sources.

The data tampering anomaly implies a slight change at the sensor level and more subtle behavior overall. For this type of attack, sensor values are intentionally altered to receive abnormal values, and the majority of network-level data remains close to normal behavior. The change for this attack type relates to the MQTT payload

size, which is increased slightly. Source IP and MAC addresses were reused from real device pairs in the normal traffic, simulating a compromised internal device.

The final step involves combining all anomalies and mixing them with the normal data into a new synthetic dataset.

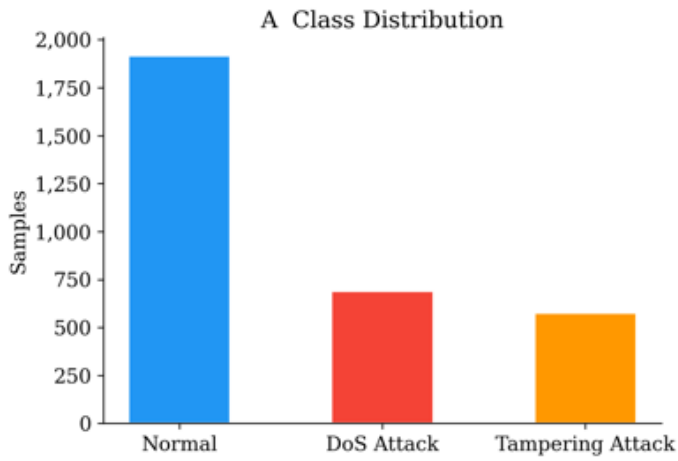


Figure 4.5: class dis.

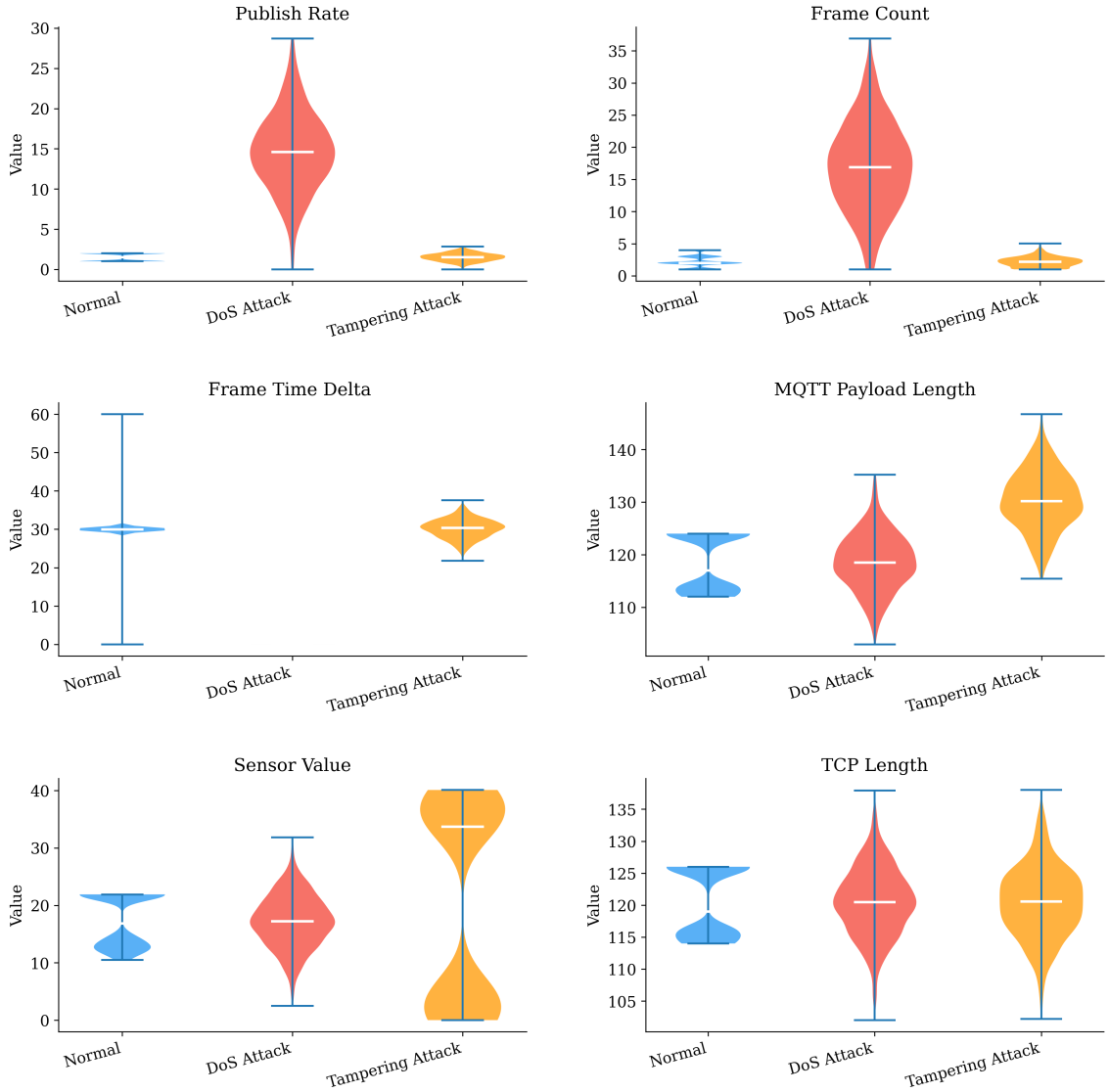


Figure 4.6: Violin plots of selected features illustrating the distribution differences between normal traffic and simulated anomalies (DoS/DDoS and data tampering).

4.7 Isolation Forest for Unsupervised Anomaly Detection

In the section, an unsupervised anomaly detection approach was chosen to identify abnormalities without relying on labeled attacks among several unsupervised methods. The Isolation Forest was selected as an anomaly detection method due to its scalability and computational efficiency. The Isolation Forest algorithm is applied to the two different datasets described in the previous sections.

4.7.1 Training Procedure

The model is trained only on the normal data, and the goal is to detect abnormal behavior in the network, allowing to learn from the patterns of the normal data; any deviation from these patterns is considered an anomaly. Missing values in categorical features are completed using mode, while missing values in numerical features are filled using the median value. Categorical variables are converted into numerical form using label encoding. dataset is divided into normal The normal data is split into training 80% and testing 20%.A MinMaxScaler is applied to scale the data between 0 and 1, fitted only on the training data, and the same scaler is then applied to the test data . The model is trained only on the normalized normal data. No attack samples are used during training. The Isolation Forest model is set with the following parameters:

Hyperparameter	Value
Max samples	1.0
Max features	0.5
Bootstrap	True
Contamination	auto
Parallel jobs	-1

Table 4.4: Isolation Forest hyperparameters configuration

4.7.2 Hyperparameter Tuning

The Isolation Forest performance highly depends on the choice of hyperparameters. In this section, many experiments were carried out in order to find the best configuration. A manual search approach was used, and different combinations of parameters were tested via experiments.

The number of trees over the following range:

$$N_{\text{estimators}} \in \{100, 150, 200, 300, 400, 450, 500, 550\}$$

Additionally, a combination of sampling, feature selection, and bootstrap was tested:

$$\text{max_samples} \in \{0.5, 1.0\}$$

$$\text{max_features} \in \{0.5, 0.75, 1.0\}$$

$$\text{bootstrap} \in \{\text{True}, \text{False}\}$$

4.7.3 Thresholding and Evaluation

The IF generates an anomaly score for each sample. The high values indicate normal samples, and the lower values indicate potential anomalies. To make a prediction, a threshold was used instead of the default contamination-based threshold in scikit-learn, which required knowing the expected proportion of anomalies in advance. In this work, a data-driven dynamic threshold was used, derived from a training set of normal data.

Procedure:

Affter train IF on the noraml data the anomaly score of these data are computed and the threshold is set as:

1. After training on scaled normal data, compute the anomaly scores of the training set.

2. Set the threshold as:

$$\theta = \mu_s - k\sigma_s \tag{4.1}$$

where:

$$\begin{aligned} \mu_s &= \text{mean of training scores} \\ \sigma_s &= \text{standard deviation of training scores} \\ k &= \text{tunable factor controlling threshold position} \end{aligned}$$

Any sample below this threshold is considered an attack, and any sample at or above it is classified as normal. This approach was chosen because it required

no prior knowledge about the attack, which is useful in real-world scenarios. The threshold is determined entirely by the data we have. This method is practical in real deployment where the proportion of attacks is unknown. The models are evaluated on the test dataset containing both Normal and attack samples. Performance is assessed using accuracy, precision, recall, and F1-score, providing a comprehensive evaluation of the model's detection capability.

Chapter 5

Results and Evaluation

5.1 MQTT/MQTTS Performance Evaluation

In this chapter, the results of experiments that measure latency, CPU time, and RAM usage of MQTT without TLS and MQTT with TLS using four different cryptographic configurations of ECC/ECDSA, X25519/Ed25519, RSA 3072-bit, and RSA 4096-bit will be presented. The experiments were conducted during CONNECT and PUBLISH operations using Raspberry Pi 4 and Mosquitto broker. The values presented are averages with error bars of standard deviation

5.1.1 Latency Results

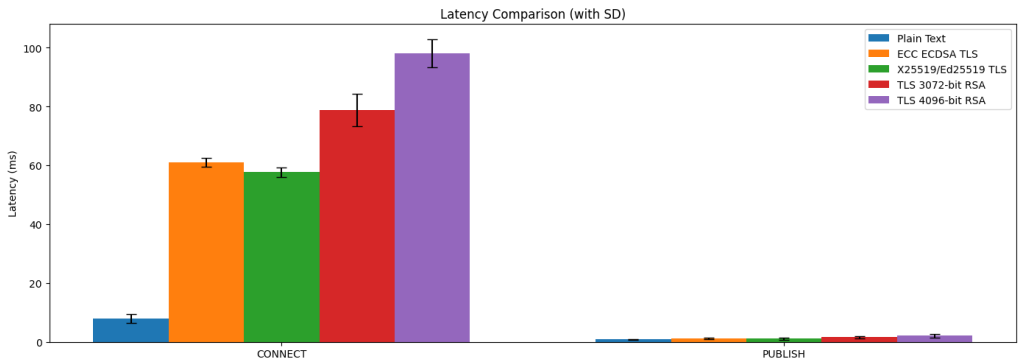


Figure 5.1: Latency comparison for CONNECT and PUBLISH operations

Figure 5.1 presents the connection and publishing latency for all the configurations during connection and publishing operations. In the connections stage, the MQTT plaintext had the lowest latency, while the TLs algorithms noticed additional delay.

The ECDSA and Ed25519 had a moderate increase in the latency, RSA 3072-bit resulted in higher latency, and the RSA 4096-bit had the highest latency due to its computationally intensive handshake process. The lowest latency between TLs algorithms was Ed25519.
For publishing operations, all configurations (MQtt and MQtt TLS) showed significant decreases in latency compared with the connect phase. The small difference between cryptographic algorithms indicates that latency is more influenced by MQTT processing rather than TLS encryption.

5.1.2 CPU Time Results

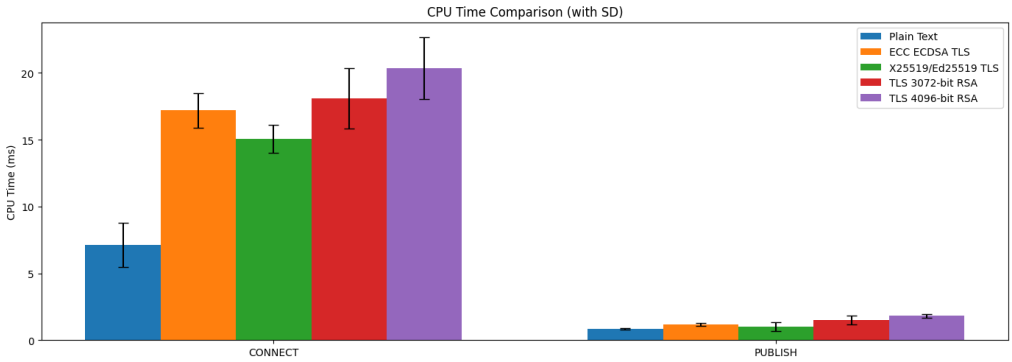


Figure 5.2: CPU time comparison (with SD) for CONNECT and PUBLISH operations

Figure 5.2 presents the connection and publishing CPU consumption for all the configurations during connection and publishing operations. The CONNECT phase required more CPU when TLS was used, and this increase followed the same pattern observed in the latency results. plaintext MQTT required minimal CPU time , ECDSA and Ed25519 showed moderate increases, RSA 3072-bit demanded more processing, and RSA 4096-bit consumed the highest CPU resources during TLS handshake execution. For publishing operations showed much lower CPU usage across all configurations. with TLS adding relatively low overhead .

5.2 Evaluation Metrics

The accuracy, F1-Score, recall, and precision metrics were applied to evaluate the performance of the proposed approach. TP indicates the number of attacks and TN indicates the number of normal traffic classified accurately. FP indicates the

number of traffics, which are actually the normal data but misclassified and regarded as attacks. FN indicates the number of attacks misclassified as normal traffic [22].

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN} \tag{5.1}$$

$$\text{Recall} = \frac{TP}{FN + TP} \tag{5.2}$$

$$\text{Precision} = \frac{TP}{FP + TP} \tag{5.3}$$

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5.4}$$

5.3 Hybrid models Performance

In this section, the performance evaluation of different machine learning models in the context of anomaly detection is presented in two different datasets. The performance evaluation of different hybrid models, including DT-FC and DT-GRU with several variants. The evaluation is based on the above metrics, shedding light on the models' capabilities in detecting anomalies.

5.3.1 Binary Classification Results For X-IIoTID Dataset

In the binary classification task on the X-IIoTID dataset, the goal was to distinguish between normal and attack network traffic . The binary classification results, as shown in Table 5.1 for the X-IIoTID dataset reveal that all models have performed exceptionally well. all models achieved nearly identical performance, with an accuracy of approximately 99.67–99.68% and a macro F1-score of about 0.9967–0.9968. This shows that all models have performed well.

Model	**Params**	**Accuracy**	**Precision**	**Recall**	**F1-Score**
FC_only	188	0.9968	0.9968	0.9967	0.9968
GRU H8	2490	0.9968	0.9968	0.9968	0.9968
GRU H16	5362	0.9967	0.9967	0.9967	0.9967
GRU H32	12258	0.9967	0.9967	0.9967	0.9967

Table 5.1: Binary classification results on the X-IIoTID (Macro) dataset with model complexity

An analysis of the training process shows in Figure 5.5 that the FC model converges gradually and reaches optimal performance after approximately 7 epochs, with early stopping triggered at epoch 10. The validation loss decreased from 0.084 at epoch 1 to approximately 0.011 at epoch 7. while validation accuracy remained stable above 99.66%. The convergence speed of GRU models is shown in Figure 5.6, The GRU models have a faster rate of convergence and achieve the optimal value within 2–3 epochs. Despite the large range in the number of trainable parameters (from 188 in the FC model up to 12,258 in the GRU h32 model), there was no measurable improvement in test performance. These indicate that, for the binary classification task, the Decision Tree leaf encoding effectively captures the discriminative structure of the data and produces highly informative features. Therefore, the contribution of the neural network models becomes limited, as even the Fully Connected (FC) model with a small number of parameters achieves performance comparable to more complex models.

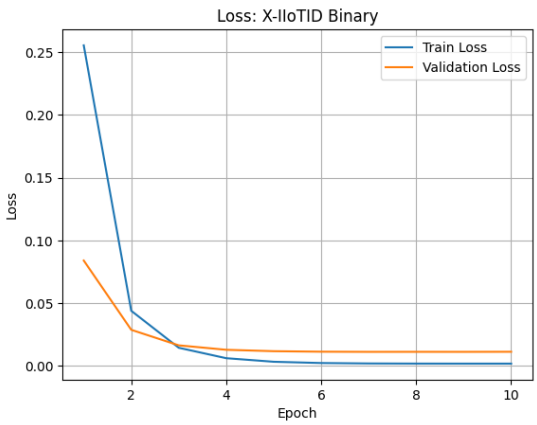


Figure 5.3: FC_only Loss

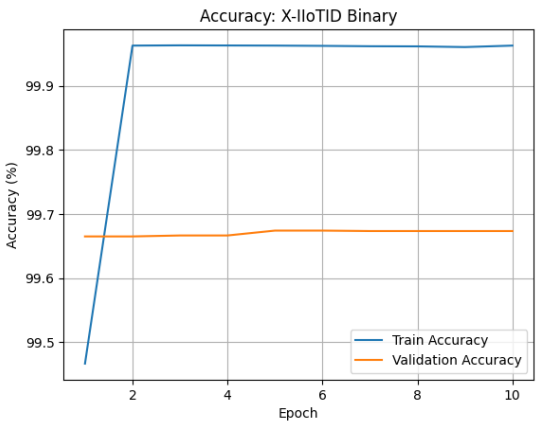
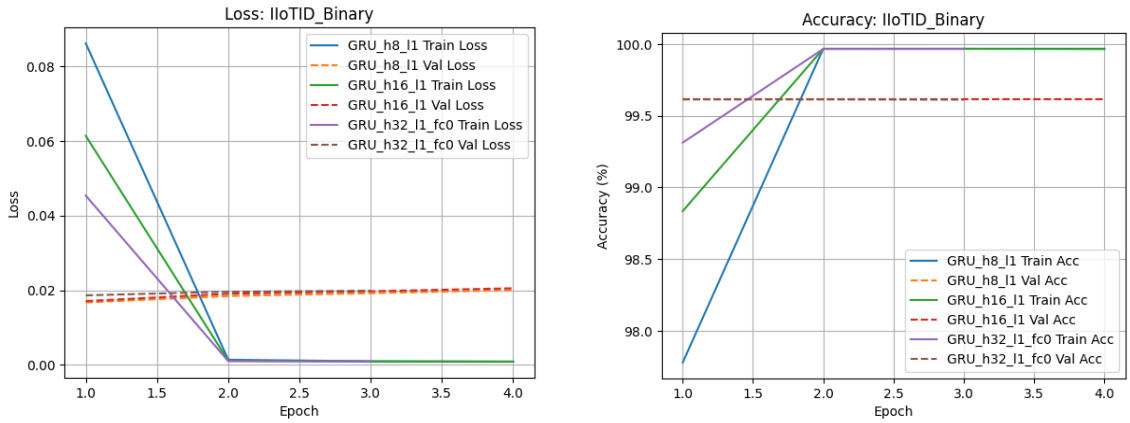


Figure 5.4: FC_only Accuracy

Figure 5.5: DT+FC training and validation curves on X-IIoTID (binary-class)



(a) Loss (b) Accuracy

Figure 5.6: DT+GRU training and validation curves on X-IIoTID (binary-class)

5.3.2 Multi-class Classification Results For X-IIoTID Dataset

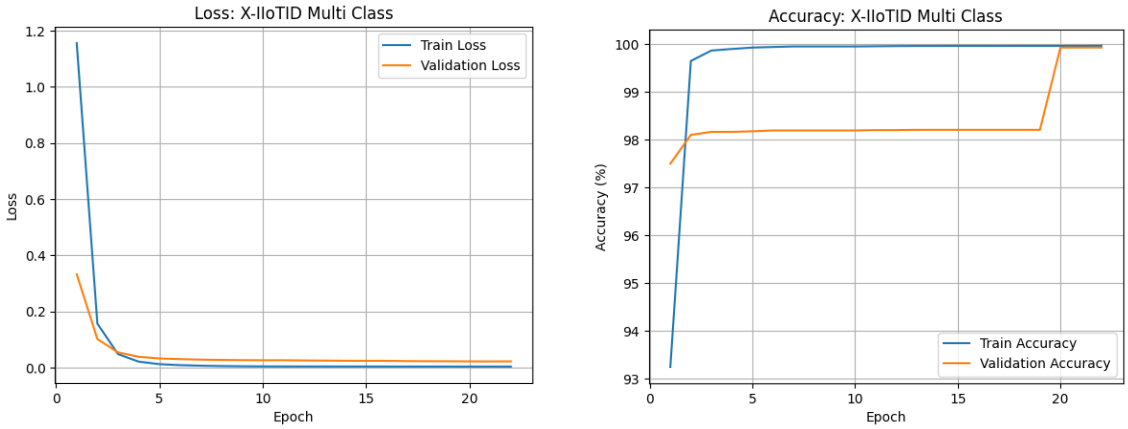
The performance of the models on the multi-class classification task using the X-IIoTID dataset, as shown in Table 5.2, shows the performance of the models in the more complex task. The FC model achieved a macro F1-score of 0.9001, while all three GRU models attained a macro F1-score of 0.9445, indicating that the GRU models are better able to exploit the feature representation generated by the Decision Tree in the multi class setting. The performance of the GRU models remains stable despite increasing the hidden size from 8 to 32 units, which means that larger model capacity does not provide additional benefit.

In contrast to the binary classification results, where all models performed similarly, the multi-class task reveals an advantage for the GRU models. This indicates that, although the Decision Tree feature extraction provides a strong representation, additional modeling capacity helps capture the finer distinctions between multiple classes. Figure 5.7 displays the learning curves of the FC model on the multi-class X-IIoTID dataset. The model converged over 22 epochs . By the final epoch, The validation loss decreased steadily, while the validation accuracy reached approximately 99.9%, indicating stable convergence of the FC model.

Figure 5.8 presents the training curves of the three GRU models on the multi-class X-IIoTID dataset.Compared to the FC model, the GRU models converge significantly faster, achieving near-optimal performance within fewer epochs.

Model	Params	Accuracy	Precision	Recall	F1-Score
FC_only	1349	0.9991	0.9469	0.8816	0.9001
GRU H8	2091	0.9998	0.9451	0.9439	0.9445
GRU H16	4547	0.9998	0.9451	0.9439	0.9445
GRU H32	10611	0.9998	0.9452	0.9438	0.9445

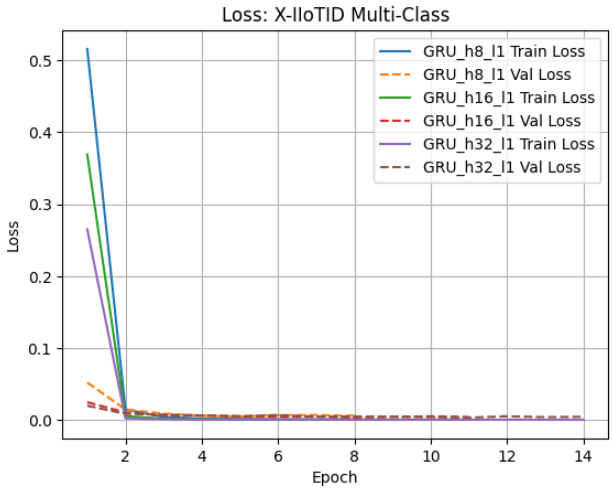
Table 5.2: Multi classification results on the X-IIoTID (Macro) dataset with model complexity



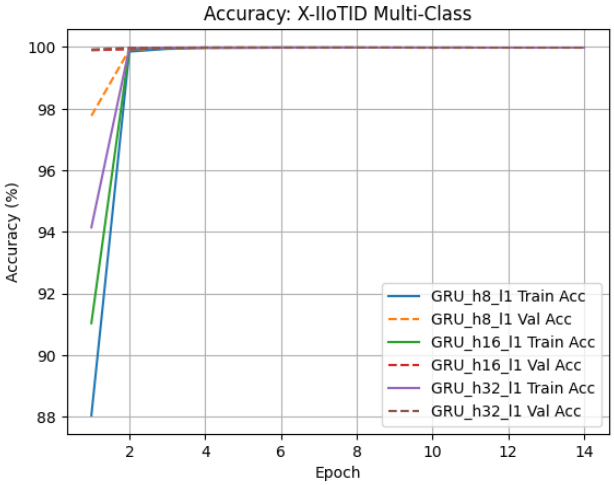
(a) Loss

(b) Accuracy

Figure 5.7: DT+FC training and validation curves on X-IIoTID (multi-class)



(a) Loss



(b) Accuracy

Figure 5.8: DT+GRU training and validation curves on X-IIoTID (multi-class)

5.3.3 Binary Classification Results For UNSW-NB15 Dataset

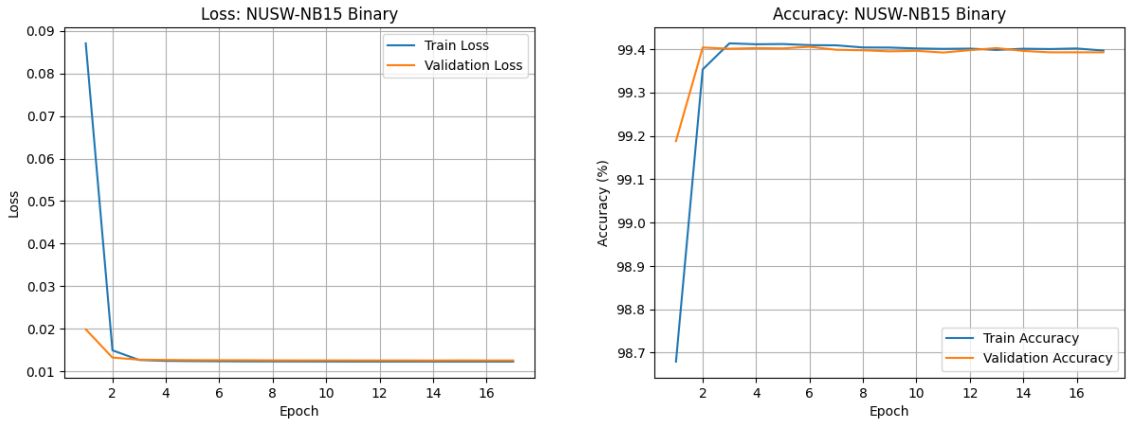
From the results shown in Table 5.3 below, the results of the binary classification using the UNSW-NB15 dataset show that all the models have achieved high classification accuracy. The accuracy of all models is nearly identical, ranging from 99.39% to 99.40%, indicating minimal variation across different architectures. Similarly, the macro F1-score shows negligible variation across models, ranging from 0.9863 to 0.9864

Despite the large increase in model complexity, from 524 parameters in the FC model to 28,386 parameters in the GRU h32 model, no significant improvement in performance is observed. This indicates that additional model capacity is unnecessary for this task. Figure 5.9 displays the training curves obtained by the FC for this task. This model contains 524 parameters and took 17 epochs before the early stopping was triggered. Both the Training and validation losses converged gradually and remained very close to each other, indicating a stable training process with good generalization and no overfitting.

Figure 5.10 displays the learning curves obtained by the three GRU models. All three models converged to an optimal value within a small number of epochs. This is due to the simplicity of the binary classification problem. This observation is similar to the case where the X-IIoTID binary classification problem was tackled by all three models.

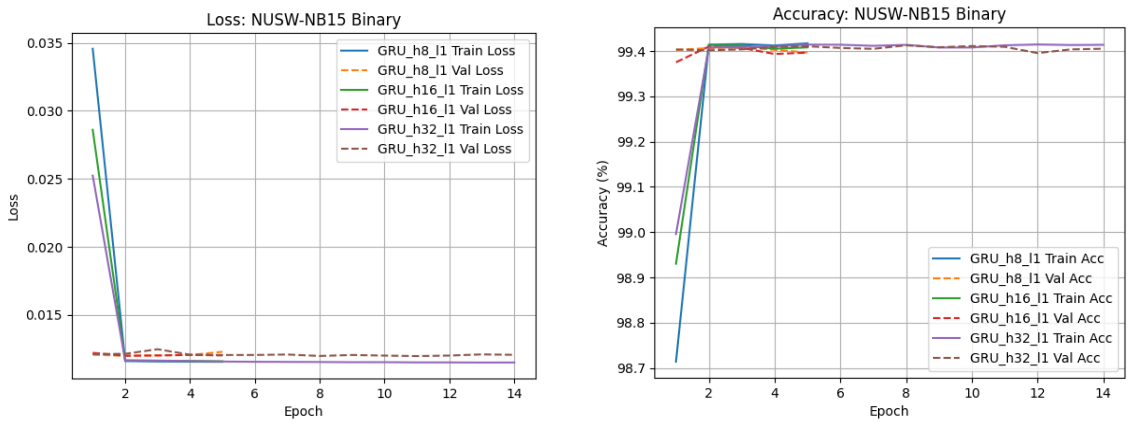
Model	**Params**	**Accuracy**	**Precision**	**Recall**	**F1-Score**
FC_only	524	0.9939	0.9859	0.9866	0.9863
GRU H8 L1	6522	0.9940	0.9876	0.9851	0.9863
GRU H16 L1	13426	0.9940	0.9876	0.9853	0.9864
GRU H32 L1	28386	0.9940	0.9871	0.9855	0.9863

Table 5.3: Binary cassification results on the UNSW15 (Macro) dataset with model complexity



(a) Loss (b) Accuracy

Figure 5.9: DT+FC training and validation curves on UNSW-NB15 (Binary-class)



(a) Loss (b) Accuracy

Figure 5.10: DT+GRU training and validation curves on UNSW-NB15 (Binary-class)

5.3.4 Multi-class Classification Results For UNSW-NB15 Dataset

The performance of the models on the multi-class classification task using the UNSW-NB15 dataset, as shown in Table 5.4, this task was the most challenging across all experiments. The FC model achieved a macro F1-score of approximately 0.5029. revealing its limited ability to distinguish between the multiple attack categories in this dataset. In general, GRU models perform better than the FC model in this task. Although the accuracy is high (around 98%), the macro F1-scores are significantly lower. This reflects severe class imbalance, where minority classes

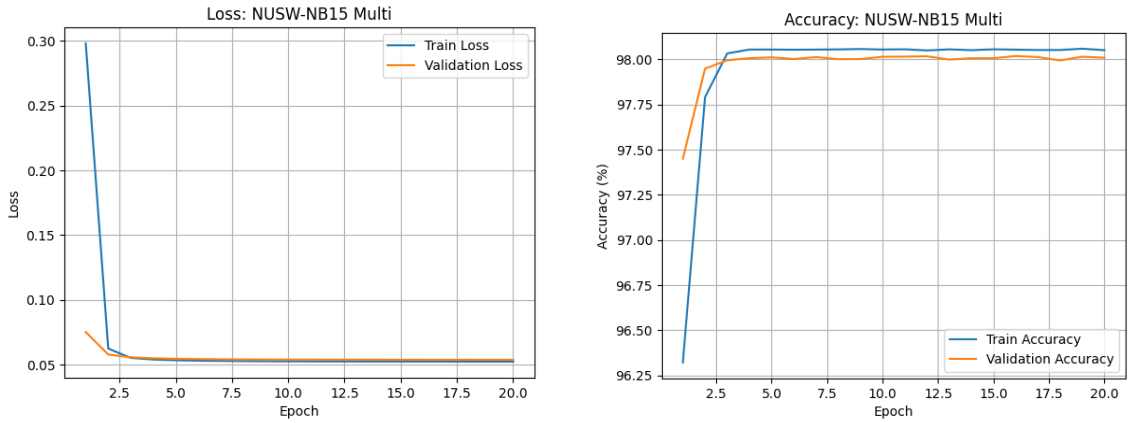
are poorly classified, making macro F1-score a more reliable metric for evaluating model performance. GRU h8 and GRU h16 achieved macro F1-scores of 0.5716 and 0.5703, respectively. However, GRU h32 performed worse compared to GRU h8 and GRU h16. The GRU model h32 contains about 47k parameters, which is a lot more compared to h8 and h16 in the imbalance dataset. This indicates that the model begins to learn noise rather than useful patterns, potentially leading to convergence to a worse local minimum.

Figure 5.11 shows the training curves for the FC model on the UNSW-NB15 multi-class task. The model trained for the full 20 epochs, with training and validation loss decreasing slowly and steadily. The slow convergence reflects the increased difficulty of this task compared to the binary case.

Figure 5.12 presents the training behavior of the GRU models on this task. The GRU models converge faster than the FC model and achieve lower validation loss values overall. Their validation losses are generally lower than those of the FC model, indicating the benefit of GRU modeling for multi-class classification. In contrast to the binary classification results, the multi-class task requires greater model capacity to distinguish between attack categories.

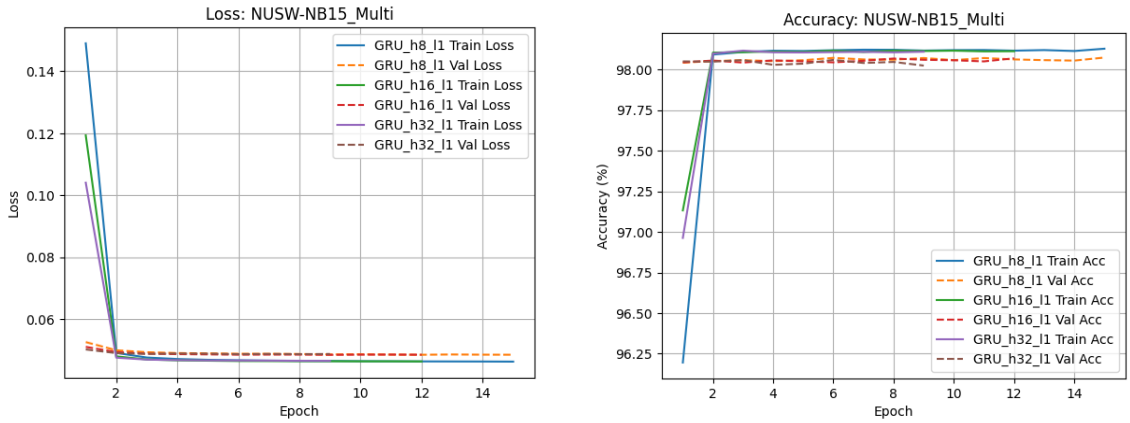
Model	**Params**	**Accuracy**	**Precision**	**Recall**	**F1-Score**
FC_only	4550	0.9800	0.5269	0.4903	0.5029
GRU H8 L1	11226	0.9806	0.7002	0.5623	0.5716
GRU H16 L1	22826	0.9807	0.7156	0.5557	0.5703
GRU H32 L1	47178	0.9806	0.6261	0.5247	0.5219

Table 5.4: Multi classification results on the UNSW15 (Macro) dataset with model complexity



(a) Loss (b) Accuracy

Figure 5.11: DT+FC training and validation curves on UNSW-NB15 (Multi-class)



(a) Loss (b) Accuracy

Figure 5.12: DT+GRU training and validation curves on UNSW-NB15 (Multi-class)

5.3.5 Comparison with Related Work

The results in Tables 5.5 and 5.6 show that the proposed models achieve high performance across both the X-IIoTID and UNSW-NB15 datasets. In binary classification, the proposed models consistently achieve very high F1-scores, reaching up to 99.68%, which are comparable to or slightly higher than existing methods.

In the multi-class task, traditional models such as SVM and LR show significantly lower performance, while deep learning models generally achieve stronger results. The proposed DT-GRU models obtain competitive performance, outperforming traditional approaches and achieving results close to advanced deep learning methods, although slightly lower than the best reported results in some cases.

In addition, the proposed hybrid models are relatively lightweight, requiring fewer parameters compared to many deep learning approaches. This makes them suitable for deployment in resource-constrained IIoT environments across different datasets.

Overall, the results demonstrate that the proposed models provide a strong balance between performance and computational efficiency.

Table 5.5: Comparison between existing models and the proposed models on UNSW-NB15 dataset

Paper / Model	**Year**	**Model**	**Binary**	**Multi**
[20]	2021	GRU	99.45% (F1)	93.54% (F1)
[20]	2021	DT	99.54% (F1)	93.80% (F1)
[20]	2021	SVM	97.88% (F1)	90.72% (F1)
[20]	2021	LR	91.45% (F1)	76.05% (F1)
[23]	2022	DL	99.79% (ACC)	– –
[24]	2021	CDAE-DNN	98.15% (F1)	97.41% (F1)
[24]	2021	AP2PFL-DNN	97.97 % (F1)	96.42 % (F1)
[24]	2021	AP2PFL-MLP	97.23 % (F1)	92.061 % (F1)
Proposed model	–	DT-FC	99.68% (F1)	90.01% (F1)
Proposed model	–	DT-GRU H8	99.68% (F1)	94.45% (F1)
Proposed model	–	DT-GRU H16	99.67% (F1)	94.45% (F1)
Proposed model	–	DT-GRU H32	99.67% (F1)	94.45% (F1)

Table 5.6: Comparison between existing models and the proposed models on UNSW-NB15 dataset

Paper / Model	**Year**	**Model**	**Binary**	**Multi**
[25]	2016	ANN	81.34% (ACC)	–
[26]	2020	LR	84.22% (F1)	66.62% (F1)
[26]	2020	SVM	71.18% (F1)	53.77% (F1)
[26]	2020	DT	90.00% (F1)	51.12% (F1)
[27]	2021	GA-RF	89.73% (F1)	–
[27]	2021	GA-ET	–	80.27% (F1)
[28]	2021	PSO-GBM	86.68% (ACC)	–
[29]	2020	NSGA2-MLR-NBTree	–	66.00% (ACC)
[29]	2020	NSGA2-BLR-DT	94.90% (ACC)	–
[30]	2022	AE-LSTM	97.55% (F1)	–
[31]	2019	DL-LSTM	–	65.9% (F1)
Proposed model	–	DT-FC	98.63% (F1)	50.29% (F1)
Proposed model	–	DT-GRU H8	98.63% (F1)	57.16% (F1)
Proposed model	–	DT-GRUH16	98.64% (F1)	57.03% (F1)
Proposed model	–	DT-GRU H32	98.63% (F1)	52.19% (F1)

5.4 IF model Performance

In this section, the performance results of an anomaly detection model based on Isolation Forest to the different datasets. In particular, the evaluation includes the performance of such model for distinguishing normal network behavior from different types of cyberattack. The experimental includes data pre-processing, model training using only normal data, threshold selection and performance evaluation using standard classification metrics

5.4.1 model Performance on X-IIoTID Dataset

The performance of the Isolation Forest model on the X-IIoTID dataset demonstrates its efficiency in separating the normal network flow from the abnormal. In terms of overall accuracy, the Isolation Forest model demonstrated 85.29% success in classifying the data into their correct category. For the normal class, the model attained a high precision value of 0.9049, recall value of 0.7974, and an F1 score of 0.8477, thus demonstrating the model's high performance at modeling the normal traffic in the network. However, when we consider the attack class, the model's precision value was 0.8100, recall value was 0.9115, while the F1 score stood at 0.8578. This indicates that the model is highly effective at detecting attacks, with a high detection rate, although it still produces some false positives.

Class	**Precision**	**Recall**	**F1-Score**
Normal	0.9049	0.7974	0.8477
Attack	0.8100	0.9115	0.8578
Macro Avg	0.8574	0.8545	0.8528
Accuracy		0.8529	

Table 5.7: Classification Performance Metrics (X-IIoTID)

The distribution of IF scores for the normal training data and the threshold used by the model is shown in Figure 5.13. The threshold is computed based on the statistical properties of the score distribution using the relation defined in Equation 4.1, where $k = 1.7$ is selected. The threshold lies within the lower tail of the distribution, allowing the model to distinguish normal behaviour from anomalous instances.

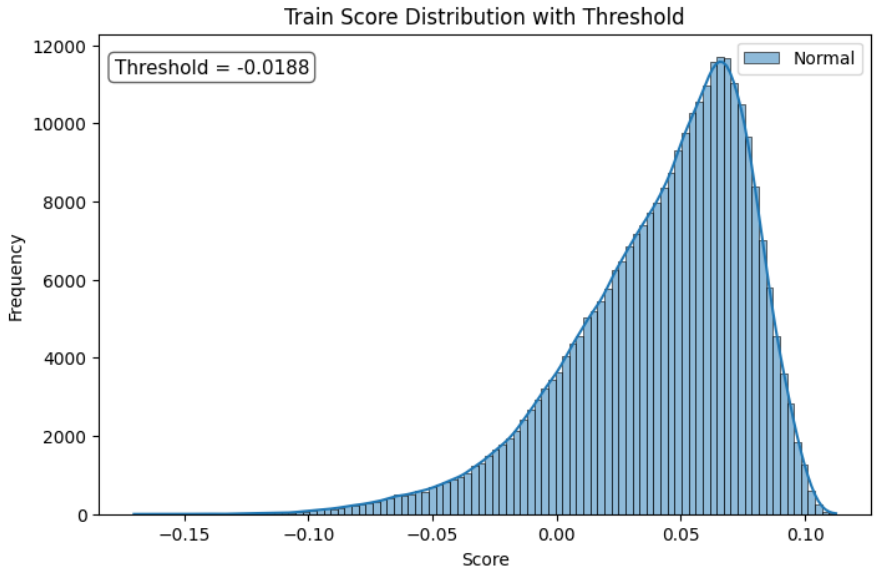


Figure 5.13: Score distribution with anomaly detection threshold (X-IIoTID)

Overall, the results above demonstrate that the Isolation Forest model provides effective anomaly detection performance, with strong precision and a reasonable level of recall, which making it suitable intrusion detection.

5.4.2 Model Performance on UNSW-NB15 Dataset

The performance of the Isolation Forest model on the UNSW-NB15 dataset further demonstrates the robustness the anomaly detection approach when applied to a different cybersecurity dataset. The model achieved an overall accuracy of 93.45%, indicating a strong ability to correctly classify both normal and attack traffic. For the attack class, the model obtained a precision of 0.6652 and an F1-score of 0.7892, while achieving a high recall of 0.9702. This indicates that the model successfully detects the majority of attack instances.

For the normal class, the model achieved a very high precision of 0.9954 and an F1-score of 0.9612, with a recall of 0.9293, confirming its effectiveness in modeling normal network behavior. The macro-average F1-score of 0.8752. further highlight the overall balanced performance of the model across both classes.

Class	**Precision**	**Recall**	**F1-Score**
Normal	0.9954	0.9293	0.9612
Attack	0.6652	0.9702	0.7892
Macro Avg	0.8303	0.9497	0.8752
Accuracy		0.9345	

Table 5.8: Classification Performance Metrics (UNSW-NB15)

The distribution of Isolation Forest scores for the normal training data, along with the selected threshold, is illustrated in Figure 5.14. The threshold is determined using the same statistical formulation described previously with $k = 1.7$. As observed, the threshold is positioned toward the lower tail of the score distribution, enabling separation between normal and anomalous samples. Compared to the X-IIoTID dataset, the distribution appears more concentrated, which contributes to the higher recall observed for attack detection.

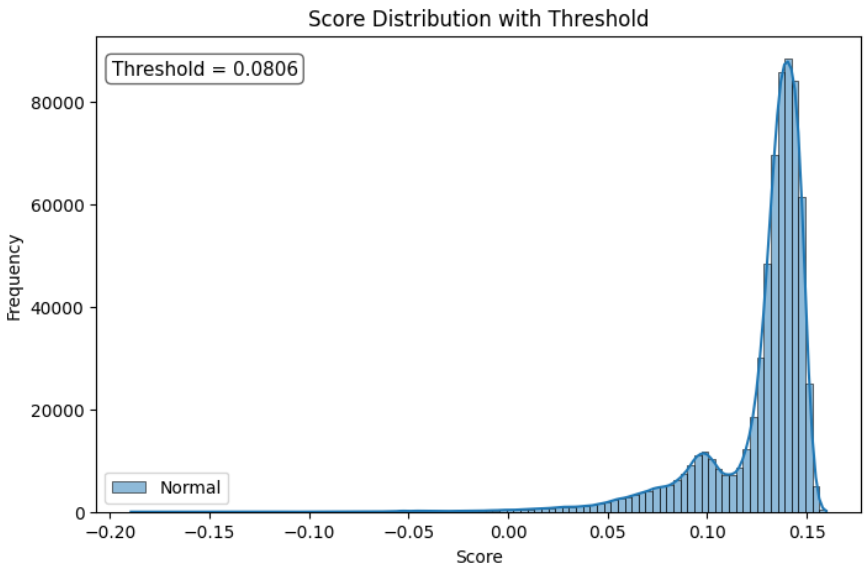


Figure 5.14: Score distribution with anomaly detection threshold (UNSW-NB15)

Overall, the results indicate that the Isolation Forest model performs well on the UNSW-NB15 dataset, achieving high attack detection capability with acceptable precision, making it suitable intrusion detection applications.

5.4.3 Model Performance on Company Dataset

Performance analysis of the Isolation Forest model on the company dataset shows that the suggested approach for IoT anomaly detection can effectively detect abnormal network traffic. With the threshold parameter k=1.7, the model reaches overall accuracy 96.95% which implies its high ability to properly distinguish both attack and normal classes.

For the attack class, the suggested approach gives precision 0.9639, F1-score 0.9782, and recall 0.9929. It is obvious that the number of detected attacks by the suggested method is close to the maximal one, with almost no false negatives.
For the normal class, the algorithm achieves precision 0.9832, F1-score 0.9496, and recall 0.9183. The results show that the majority of normal instances are identified and classified without mistakes. The high macro-average F1-score 0.9639 emphasizes the good overall performance of the suggested model for IoT anomaly detection. Comparing the performance of the algorithm on the company and UNSW-NB15 datasets, it is possible to state that the former demonstrates higher overall accuracy, better precision-recall trade-off, especially concerning the identification of attacks. Figure 5.15 provides information about the score distribution for the training normal data with the threshold. The threshold is chosen according to the same formula with

k=1.7.

Class	Precision	Recall	F1-Score
Normal	0.9832	0.9183	0.9496
Attack	0.9639	0.9929	0.9782
Macro Avg	0.9736	0.9556	0.9639
Accuracy		0.9695	

Table 5.9: Classification Performance Metrics (Company Dataset)

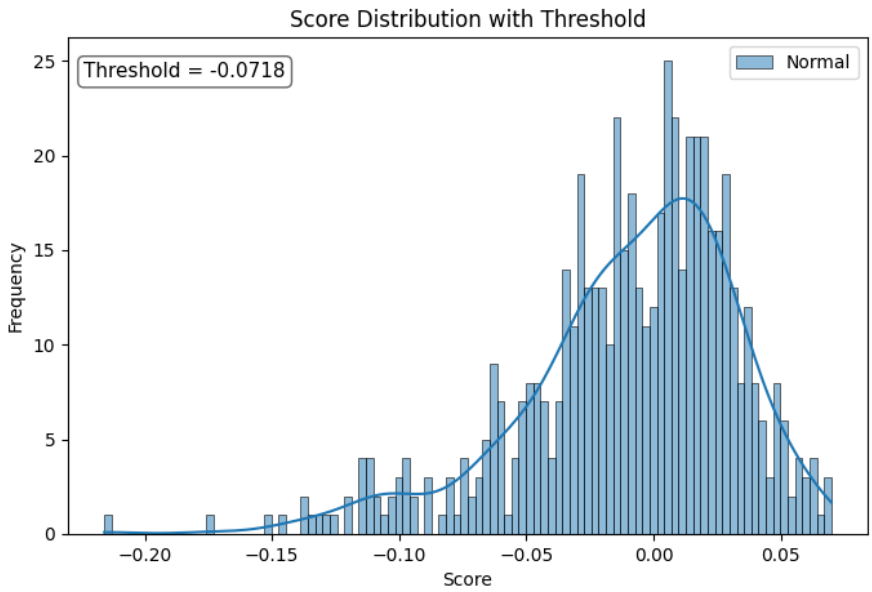


Figure 5.15: Score distribution with anomaly detection threshold (Company Dataset)

Overall, the results indicate that the Isolation Forest model performs very effectively on the company dataset, achieving high detection capability with strong precision and recall.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This thesis addressed secure MQTT communication through a TLS-based evaluation, as well as anomaly detection in IIoT environments using machine learning techniques, including both supervised hybrid models and an unsupervised Isolation Forest framework.

The results of the MQTT/MQTTS provided comparative analysis of four cryptographic configurations (ECC/ECDSA, X25519/Ed25519, RSA 3072-bit, and RSA 4096-bit) across both connection and publishing phases on a Raspberry Pi 4. The evaluation showed that using TLS adds a significant overhead in terms of response time and CPU consumption, especially during the connection establishment phase. The results show that RSA 4096-bit imposed the highest computational cost, while lighter Ed25519 algorithm demonstrated better performance in terms of reducing latency and resource consumption, offering balance between security strength and resource consumption. The publishing phase showed that once a TLS session is established, the encryption overhead becomes negligible across all algorithms, the main overhead of encryption is concentrated in the initial handshake phase. Unlike many previous studies, this thesis conducted analyzes the overhead introduced by TLS and identifies which parts consume the most resources. providing a more holistic assessment of the impact of TLS in IoT environments.

As for the hybrid machine learning models (DT-FC and DT-GRU), the results demonstrated high performance on the X-IIoTID and UNSW-NB15 datasets. In binary classification tasks, all models achieved very close results. This indicates that the decision tree leaf representations provide highly discriminative features, reducing the need for additional sequential modeling when separating the two classes. In con-

trast, in multi-class classification, GRU models consistently outperformed the FC baseline. Notably, increasing model size does not necessarily lead to improved performance; in fact, it can sometimes lead to worse results, suggesting that lightweight models achieve the best balance between performance and complexity.

As for the Isolation Forest model, it has proven effective in detecting anomalies across various datasets, demonstrating a high ability to detect attacks. One of the significant contributions of this thesis is the proposal of a dynamic mechanism for determining the decision threshold based on the distribution of anomaly scores in the normal data, rather than relying on the contamination rate, which requires prior knowledge of the attack rate, or fix threshould, which making it unable to adapt to different datasets or when shifts in the data traffic distribution.
This approach makes the system more suitable for real-world applications where labeled data is not available. Furthermore, the tuning parameter k allows for controlling the balance between false positives and false negatives according to the application's needs.

Overall, the results of this thesis demonstrate the potential for an effective and secure anomaly detection system in smart water networks by combining lightweight encryption techniques, high-performance hybrid models, and unsupervised detection methods. This system provides a practical balance between performance, accuracy, and resource consumption, making it suitable for application in industrial Internet of Things environments. Future work could include expanding the evaluation to include additional real-world data, testing the models in real-time operational environments, and developing more advanced adaptive mechanisms to improve detection accuracy in the face of changing data patterns.

6.2 Future Work

Although this thesis provides a comprehensive evaluation of secure MQTT communication and anomaly detection in IIoT environments, several directions can be explored in future work:
First, the proposed system can be extended to support real-time deployment in edge and fog computing environments. Implementing the models on edge devices would allow faster detection and reduced latency, which is critical for time-sensitive industrial applications.
Second, the anomaly detection models can be enhanced by addressing class imbal-

ance more effectively. Techniques such as data augmentation, cost-sensitive learning, or advanced sampling methods may improve the detection performance for minority attack classes, particularly in challenging datasets such as UNSW-NB15.
Third, the proposed dynamic thresholding mechanism for the Isolation Forest model can be further extended to adapt automatically over time. Incorporating adaptive or online learning strategies would allow the threshold to adjust to concept drift and changes in network traffic patterns.
Fourth, future work may explore the integration of other deep learning architectures, such as attention-based models, to improve multi-class classification performance.
In addition, This study focused only on MQTT; therefore, future work should extend the evaluation to other IoT communication protocols such as CoAP and DDS, in order to generalize the findings across different systems. Additionally, further research should consider real-world scalability, particularly in cloud-based environments where MQTT brokers are typically deployed, by investigating large-scale scenarios and their impact on performance and resource usage.

Bibliography

[1] Andrew and R. Gupta, "Mqtt version 3.1.1," OASIS Standard, Technical Standard, 2014.

[2] T. Dierks and C. Allen, "The tls protocol," Internet Engineering Task Force (IETF), Standards Track RFC RFC 2246, Jan. 1999. [Online]. Available: `https://www.ietf.org/rfc/rfc2246.txt`.

[3] Dierks and Rescorla, "The transport layer security (tls) protocol version 1.2," Standards Track RFC, Tech. Rep. RFC 5246, Aug. 2008.

[4] B. Schneier, *Applied Cryptography (2nd Edition)*. John Wiley & Sons Inc., 1996.

[5] T. Dierks and E. Rescorla, "The transport layer security (tls) protocol version 1.2," Network Working Group, RFC 5246, 2008. [Online]. Available: `https://www.rfc-editor.org/rfc/rfc5246.txt`.

[6] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.

[7] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th. Pearson, 2017.

[8] V. S. Miller, "Use of elliptic curves in cryptography," *Exploratory Computer Science*, 1998.

[9] N. Koblitz, "Elliptic curve cryptosystems," *Mathematics of computation*, vol. 48, no. 177, pp. 203–209, 1987.

[10] D. J. Bernstein, "Curve25519: New diffie-hellman speed records," in *International workshop on public key cryptography*, Springer, 2006, pp. 207–228.

[11] E. Rescorla, *The transport layer security (tls) protocol version 1.3*, `https://datatracker.ietf.org/doc/html/rfc8446`, RFC 8446, 2018.

[12] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *Journal of cryptographic engineering*, vol. 2, no. 2, pp. 77–89, 2012.

[13] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Transactions on Knowledge Discovery from Data (TKDD)*, vol. 6, no. 1, pp. 1–39, 2012.

[14] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 Eighth IEEE International Conference on Data Mining*, 2008, pp. 413–422. DOI: `10.1109/ICDM.2008.17`.

[15] Scikit-learn Developers, *Isolationforest - scikit-learn 0.18 documentation*, Accessed: 2023-XX-XX, 2016. [Online]. Available: `https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html`.

[16] K. Cho et al., "Learning phrase representations using rnn encoder–decoder for statistical machine translation," in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 2014, pp. 1724–1734.

[17] G. Shen, Q. Tan, H. Zhang, P. Zeng, and J. Xu, "Deep learning with gated recurrent unit networks for financial sequence predictions," *Procedia computer science*, vol. 131, pp. 895–903, 2018.

[18] Z.-H. Zhou, *Ensemble Methods: Foundations and Algorithms*. CRC Press, 2012.

[19] N. Moustafa and J. Slay, "Unsw-nb15: A comprehensive data set for network intrusion detection systems (unsw-nb15 network data set)," in *2015 military communications and information systems conference (MilCIS)*, Ieee, 2015, pp. 1–6.

[20] M. Al-Hawawreh, E. Sitnikova, and N. Aboutorab, "X-iiotid: A connectivity-agnostic and device-agnostic intrusion data set for industrial internet of things," *IEEE Internet of Things Journal*, vol. 9, no. 5, pp. 3962–3977, 2021.

[21] T. Sasi, A. H. Lashkari, R. Lu, P. Xiong, and S. Iqbal, "A comprehensive survey on iot attacks: Taxonomy, detection mechanisms and challenges," *Journal of Information and intelligence*, vol. 2, no. 6, pp. 455–513, 2024.

[22] H. C. Altunay and Z. Albayrak, "A hybrid cnn+ lstm-based intrusion detection system for industrial iot networks," *Engineering Science and Technology, an International Journal*, vol. 38, p. 101 322, 2023.

[23] A. Makkar, T. W. Kim, A. K. Singh, J. Kang, and J. H. Park, “Secureiiot environment: Federated learning empowered approach for securing iiot from data breach,” *IEEE Transactions on Industrial Informatics*, vol. 18, no. 9, pp. 6406–6414, 2022.

[24] M. Al-Hawawreh, E. Sitnikova, and N. Aboutorab, “Asynchronous peer-to-peer federated capability-based targeted ransomware detection model for industrial iot,” *IEEE Access*, vol. 9, pp. 148 738–148 755, 2021.

[25] N. Moustafa and J. Slay, “The evaluation of network anomaly detection systems: Statistical analysis of the unsw-nb15 data set and the comparison with the kdd99 data set,” *Information Security Journal: A Global Perspective*, vol. 25, no. 1-3, pp. 18–31, 2016.

[26] S. M. Kasongo and Y. Sun, “Performance analysis of intrusion detection systems using a feature selection method on the unsw-nb15 dataset,” *Journal of Big Data*, vol. 7, no. 1, p. 105, 2020.

[27] S. M. Kasongo, “An advanced intrusion detection system for iiot based on ga and tree based algorithms,” *IEEe Access*, vol. 9, pp. 113 199–113 212, 2021.

[28] J. Liu, D. Yang, M. Lian, and M. Li, “Research on intrusion detection based on particle swarm optimization in iot,” *Ieee Access*, vol. 9, pp. 38 254–38 268, 2021.

[29] C. Khammassi and S. Krichen, “A nsga2-lr wrapper approach for feature selection in network intrusion detection,” *Computer Networks*, vol. 172, p. 107 183, 2020.

[30] I. A. Khan, M. Keshk, D. Pi, N. Khan, Y. Hussain, and H. Soliman, “Enhancing iiot networks protection: A robust security model for attack detection in internet industrial control systems,” *Ad Hoc Networks*, vol. 134, p. 102 930, 2022.

[31] V. E. Adeyemo, A. Abdullah, N. JhanJhi, M. Supramaniam, and A. O. Balogun, “Ensemble and deep-learning methods for two-class and multi-attack anomaly intrusion detection: An empirical study,” *International Journal of Advanced Computer Science and Applications*, vol. 10, no. 9, 2019.

[1] S. Shapsough, F. Aloul, and I. A. Zualkernan, "Securing low-resource edge devices for IoT systems," in 2018 International Symposium in Sensing and Instrumentation in IoT Era (ISSI), 2018, pp. 1–4.

[2] H.-Y. Chien, P.-C. Lin, and M.-L. Chiang, "Efficient MQTT Platform Facilitating Secure Group Communication," J. Internet Technol., vol. 21, no. 7, pp.

1929–1940, 2020.

[3] P. Urien, "A New IoT Trust Model Based on TLS-SE and TLS-IM Secure Elements: A Blockchain Use Case," in 2021 IEEE 18th Annual Consumer Communications Networking Conference (CCNC), 2021, pp. 1–2.